

```
In [1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import os
import seaborn as sns
```

```
In [5]: project_path="Sales_Project"
os.makedirs(project_path,exist_ok=True)
print(project_path,"created sucessfully")
```

Sales_Project created sucessfully

```
In [16]: import pandas as pd

coustmer = pd.read_csv(r"C:\Users\yaswa\Desktop\coustmer.csv")

print(coustmer.head())
```

	customer_id	customer_name	Age	Gender	City	Member_ship
0	C1001	Venky	19	NaN	guntur	gold
1	C1002	teja	20	NaN	hydhrabad	sliver
2	C1003	lakshman	16	NaN	nalagonda	platinum
3	C1004	lokesh	19	NaN	guntur	platinum
4	C1005	harish	30	NaN	pidugralla	silver

```
In [17]: print(coustmer)
```

	customer_id	customer_name	Age	Gender	City	Member_ship
0	C1001	Venky	19	NaN	guntur	gold
1	C1002	teja	20	NaN	hydhrabad	sliver
2	C1003	lakshman	16	NaN	nalagonda	platinum
3	C1004	lokesh	19	NaN	guntur	platinum
4	C1005	harish	30	NaN	pidugralla	silver
5	C1006	gopi	32	NaN	vijaywada	silver
6	C1007	ram	32	NaN	guntur	gold
7	C1008	charan	26	NaN	guntur	gold
8	C1009	ravi	21	NaN	hydhrabad	sliver
9	C1010	teja	17	NaN	nalagonda	platinum
10	C1011	afrid	12	NaN	guntur	platinum
11	C1012	raj	22	NaN	pidugralla	silver
12	C1013	kumar	27	NaN	vijaywada	silver
13	C1014	shiva	28	NaN	guntur	gold
14	C1015	sai	39	NaN	guntur	gold
15	C1016	surya	50	NaN	hydhrabad	sliver
16	C1017	hemanth	32	NaN	nalagonda	platinum
17	C1018	imran	43	NaN	guntur	platinum
18	C1019	rahul	54	NaN	dachepalli	silver
19	C1020	aditya	21	NaN	vijaywada	silver
20	C1021	umesh	24	NaN	guntur	gold
21	C1022	karthik	25	NaN	guntur	gold
22	C1023	bushan	28	NaN	hydhrabad	sliver
23	C1024	sudheer	10	NaN	nalagonda	platinum
24	C1025	maruthi	11	NaN	guntur	platinum
25	C1026	vijay	19	NaN	tenali	silver

```
In [23]: product = pd.read_csv(r"C:\Users\yaswa\Desktop\product.csv")
print(coustmer.head())
```

	customer_id	customer_name	Age	Gender	City	Member_ship
0	C1001	Venky	19	NaN	guntur	gold
1	C1002	teja	20	NaN	hydhrabad	sliver
2	C1003	lakshman	16	NaN	nalagonda	platinum
3	C1004	lokes	19	NaN	guntur	platinum
4	C1005	harish	30	NaN	pidugralla	silver

```
In [26]: Transaction = pd.read_csv(r"C:\Users\yaswa\Desktop\Transection.csv")
print(coustmer.head())
```

	customer_id	customer_name	Age	Gender	City	Member_ship
0	C1001	Venky	19	NaN	guntur	gold
1	C1002	teja	20	NaN	hydhrabad	sliver
2	C1003	lakshman	16	NaN	nalagonda	platinum
3	C1004	lokes	19	NaN	guntur	platinum
4	C1005	harish	30	NaN	pidugralla	silver

```
In [27]: project_path = r"C:\Users\yaswa\Desktop"
coustmer.to_csv(os.path.join(project_path, "coustmer.csv"), index=False)
product.to_csv(os.path.join(project_path, "product.csv"), index=False)
Transaction.to_csv(os.path.join(project_path, "Transaction.csv"), index=False)
print("Raw datasets saved successfully")
```

Raw datasets saved successfully

```
In [31]: print(Transaction.columns)
print(coustmer.columns)
print(product.columns)
```

```
Index(['Transaction_id', 'Customer_id', 'Product_id', 'Quality', 'Discount',
       'Payment_mode', 'Sale_data'],
      dtype='object')
```

```
Index(['customer_id', 'customer_name', 'Age', 'Gender', 'City', 'Member_ship'],
      dtype='object')
```

```
Index(['Product_id', 'Product_name', 'Category', 'Unit_price'], dtype='object')
```

```
In [36]: df = Transaction.merge(coustmer, on="Customer_id")

df = df.merge(product, on="Product_id")

print("Merged successfully")
```

Merged successfully

```
In [41]: df.head()
```

```
Out[41]:
```

Transaction_id	Customer_id	Product_id	Quality	Discount	Payment_mode	Sale_data
----------------	-------------	------------	---------	----------	--------------	-----------

◀ [Progress Bar] ▶

```
In [42]: df.tail()
```

```
Out[42]:
```

Transaction_id	Customer_id	Product_id	Quality	Discount	Payment_mode	Sale_data
----------------	-------------	------------	---------	----------	--------------	-----------

◀ [Progress Bar] ▶

```
In [40]: df.shape
```

```
Out[40]: (0, 15)
```

In [43]: `df.info()`

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 0 entries
Data columns (total 15 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Transaction_id         0 non-null     object
1   Customer_id            0 non-null     object
2   Product_id             0 non-null     object
3   Quality                0 non-null     int64
4   Discount               0 non-null     int64
5   Payment_mode           0 non-null     object
6   Sale_data              0 non-null     object
7   customer_name          0 non-null     object
8   Age                    0 non-null     int64
9   Gender                 0 non-null     float64
10  City                   0 non-null     object
11  Member_ship            0 non-null     object
12  Product_name           0 non-null     object
13  Category               0 non-null     object
14  Unit_price             0 non-null     float64
dtypes: float64(2), int64(3), object(10)
memory usage: 132.0+ bytes
```

In [45]: `df.isnull().sum()`

```
Out[45]: Transaction_id    0
         Customer_id      0
         Product_id       0
         Quality          0
         Discount         0
         Payment_mode     0
         Sale_data        0
         customer_name    0
         Age              0
         Gender           0
         City             0
         Member_ship      0
         Product_name     0
         Category         0
         Unit_price       0
         dtype: int64
```

In [46]: `df.columns`

```
Out[46]: Index(['Transaction_id', 'Customer_id', 'Product_id', 'Quality', 'Discount',
               'Payment_mode', 'Sale_data', 'customer_name', 'Age', 'Gender', 'City',
               'Member_ship', 'Product_name', 'Category', 'Unit_price'],
              dtype='object')
```

In [50]: `print(df.shape)`
`print(df.head())`

```
(0, 15)
Empty DataFrame
Columns: [Transaction_id, Customer_id, Product_id, Quality, Discount, Payment_mod
e, Sale_data, customer_name, Age, Gender, City, Member_ship, Product_name, Catego
ry, Unit_price]
Index: []
```

```
In [51]: print("Transaction:", Transaction.shape)
print("Customer:", coustmer.shape)
print("Product:", product.shape)
```

```
Transaction: (25, 7)
Customer: (26, 6)
Product: (25, 4)
```

```
In [52]: print(Transaction["Customer_id"].head())
print(coustmer["Customer_id"].head())

print(Transaction["Product_id"].head())
print(product["Product_id"].head())
```

```
0    C001
1    C002
2    C003
3    C004
4    C005
Name: Customer_id, dtype: object
0    C1001
1    C1002
2    C1003
3    C1004
4    C1005
Name: Customer_id, dtype: object
0    P001
1    P002
2    P003
3    P004
4    P005
Name: Product_id, dtype: object
0    P001
1    P002
2    P003
3    P004
4    P005
Name: Product_id, dtype: object
```

```
In [54]: print(Transaction["Customer_id"].isin(coustmer["Customer_id"]).sum())
```

```
0
```

```
In [55]: coustmer.rename(columns={"customer_id": "Customer_id"}, inplace=True)

df = Transaction.merge(
    coustmer,
    on="Customer_id",
    how="left"
)

df = df.merge(
    product,
    on="Product_id",
    how="left"
)

print("Merged successfully")
print(df.shape)
print(df.head())
```

Merged successfully

(25, 15)

	Transaction_id	Customer_id	Product_id	Quality	Discount	Payment_mode	\
0	T001	C001	P001	2	5	UPI	
1	T002	C002	P002	3	10	Cash	
2	T003	C003	P003	1	0	Card	
3	T004	C004	P004	2	15	UPI	
4	T005	C005	P005	1	5	Cash	

	Sale_data	customer_name	Age	Gender	City	Member_ship	Product_name	\
0	01-07-2026	NaN	NaN	NaN	NaN	NaN	Rice	
1	02-07-2026	NaN	NaN	NaN	NaN	NaN	Milk	
2	03-07-2026	NaN	NaN	NaN	NaN	NaN	Honey	
3	04-07-2026	NaN	NaN	NaN	NaN	NaN	Ghee	
4	05-07-2026	NaN	NaN	NaN	NaN	NaN	Shampoo	

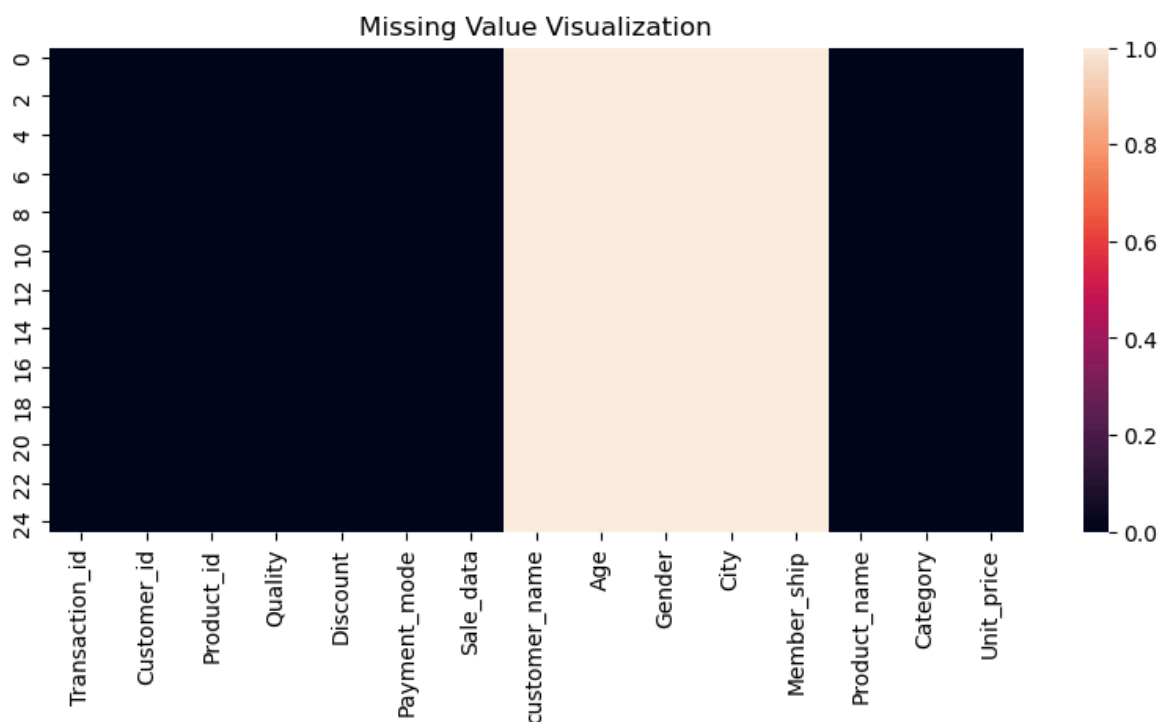
	Category	Unit_price
0	Groceries	60.0
1	Dairy	55.0
2	Groceries	250.0
3	Dairy	650.0
4	Personal Care	180.0

```
In [56]: import matplotlib.pyplot as plt
import seaborn as sns

plt.figure(figsize=(10, 4))

sns.heatmap(df.isnull(), cbar=True)

plt.title("Missing Value Visualization")
plt.show()
```



```
In [3]: !pip install ydata-profiling
```

Requirement already satisfied: ydata-profiling in c:\users\yaswa\anaconda3\lib\site-packages (4.18.4)

Requirement already satisfied: scipy<1.17,>=1.8 in c:\users\yaswa\anaconda3\lib\site-packages (from ydata-profiling) (1.13.1)

Requirement already satisfied: pandas!=1.4.0,<3.0,>1.5 in c:\users\yaswa\anaconda3\lib\site-packages (from ydata-profiling) (2.2.2)

Requirement already satisfied: matplotlib<=3.10,>=3.5 in c:\users\yaswa\anaconda3\lib\site-packages (from ydata-profiling) (3.9.2)

Requirement already satisfied: pydantic<3,>=2 in c:\users\yaswa\anaconda3\lib\site-packages (from ydata-profiling) (2.8.2)

Requirement already satisfied: PyYAML<6.1,>=6.0.3 in c:\users\yaswa\anaconda3\lib\site-packages (from ydata-profiling) (6.0.3)

Requirement already satisfied: jinja2<3.2,>=3.1.6 in c:\users\yaswa\anaconda3\lib\site-packages (from ydata-profiling) (3.1.6)

Requirement already satisfied: visions<0.8.2,>=0.7.5 in c:\users\yaswa\anaconda3\lib\site-packages (from visions[type_image_path]<0.8.2,>=0.7.5->ydata-profiling) (0.8.1)

Requirement already satisfied: numpy<2.4,>=1.22 in c:\users\yaswa\anaconda3\lib\site-packages (from ydata-profiling) (1.26.4)

Requirement already satisfied: minify-html>=0.15.0 in c:\users\yaswa\anaconda3\lib\site-packages (from ydata-profiling) (0.18.1)

Requirement already satisfied: filetype>=1.0.0 in c:\users\yaswa\anaconda3\lib\site-packages (from ydata-profiling) (1.2.0)

Requirement already satisfied: phik<0.13,>=0.12.5 in c:\users\yaswa\anaconda3\lib\site-packages (from ydata-profiling) (0.12.5)

Requirement already satisfied: requests<3,>=2.32.0 in c:\users\yaswa\anaconda3\lib\site-packages (from ydata-profiling) (2.32.3)

Requirement already satisfied: tqdm<5,>=4.66.3 in c:\users\yaswa\anaconda3\lib\site-packages (from ydata-profiling) (4.66.5)

Requirement already satisfied: seaborn<0.14,>=0.10.1 in c:\users\yaswa\anaconda3\lib\site-packages (from ydata-profiling) (0.13.2)

Requirement already satisfied: multimethod<2,>=1.4 in c:\users\yaswa\anaconda3\lib\site-packages (from ydata-profiling) (1.12)

Requirement already satisfied: statsmodels<1,>=0.13.2 in c:\users\yaswa\anaconda3\lib\site-packages (from ydata-profiling) (0.14.2)

Requirement already satisfied: typeguard<5,>=4 in c:\users\yaswa\anaconda3\lib\site-packages (from ydata-profiling) (4.6.0)

Requirement already satisfied: imagehash==4.3.2 in c:\users\yaswa\anaconda3\lib\site-packages (from ydata-profiling) (4.3.2)

Requirement already satisfied: wordcloud>=1.9.4 in c:\users\yaswa\anaconda3\lib\site-packages (from ydata-profiling) (1.9.6)

Requirement already satisfied: dacite<2,>=1.9 in c:\users\yaswa\anaconda3\lib\site-packages (from ydata-profiling) (1.9.2)

Requirement already satisfied: numba<0.63,>=0.60 in c:\users\yaswa\anaconda3\lib\site-packages (from ydata-profiling) (0.60.0)

Requirement already satisfied: PyWavelets in c:\users\yaswa\anaconda3\lib\site-packages (from imagehash==4.3.2->ydata-profiling) (1.7.0)

Requirement already satisfied: pillow in c:\users\yaswa\anaconda3\lib\site-packages (from imagehash==4.3.2->ydata-profiling) (10.4.0)

Requirement already satisfied: MarkupSafe>=2.0 in c:\users\yaswa\anaconda3\lib\site-packages (from jinja2<3.2,>=3.1.6->ydata-profiling) (2.1.3)

Requirement already satisfied: contourpy>=1.0.1 in c:\users\yaswa\anaconda3\lib\site-packages (from matplotlib<=3.10,>=3.5->ydata-profiling) (1.2.0)

Requirement already satisfied: cycycler>=0.10 in c:\users\yaswa\anaconda3\lib\site-packages (from matplotlib<=3.10,>=3.5->ydata-profiling) (0.11.0)

Requirement already satisfied: fonttools>=4.22.0 in c:\users\yaswa\anaconda3\lib\site-packages (from matplotlib<=3.10,>=3.5->ydata-profiling) (4.51.0)

Requirement already satisfied: kiwisolver>=1.3.1 in c:\users\yaswa\anaconda3\lib\site-packages (from matplotlib<=3.10,>=3.5->ydata-profiling) (1.4.4)

Requirement already satisfied: packaging>=20.0 in c:\users\yaswa\anaconda3\lib\si

te-packages (from matplotlib<=3.10,>=3.5->ydata-profiling) (24.1)
 Requirement already satisfied: pyparsing>=2.3.1 in c:\users\yaswa\anaconda3\lib\site-packages (from matplotlib<=3.10,>=3.5->ydata-profiling) (3.1.2)
 Requirement already satisfied: python-dateutil>=2.7 in c:\users\yaswa\anaconda3\lib\site-packages (from matplotlib<=3.10,>=3.5->ydata-profiling) (2.9.0.post0)
 Requirement already satisfied: llvmlite<0.44,>=0.43.0dev0 in c:\users\yaswa\anaconda3\lib\site-packages (from numba<0.63,>=0.60->ydata-profiling) (0.43.0)
 Requirement already satisfied: pytz>=2020.1 in c:\users\yaswa\anaconda3\lib\site-packages (from pandas!=1.4.0,<3.0,>1.5->ydata-profiling) (2024.1)
 Requirement already satisfied: tzdata>=2022.7 in c:\users\yaswa\anaconda3\lib\site-packages (from pandas!=1.4.0,<3.0,>1.5->ydata-profiling) (2023.3)
 Requirement already satisfied: joblib>=0.14.1 in c:\users\yaswa\anaconda3\lib\site-packages (from phik<0.13,>=0.12.5->ydata-profiling) (1.4.2)
 Requirement already satisfied: annotated-types>=0.4.0 in c:\users\yaswa\anaconda3\lib\site-packages (from pydantic<3,>=2->ydata-profiling) (0.6.0)
 Requirement already satisfied: pydantic-core==2.20.1 in c:\users\yaswa\anaconda3\lib\site-packages (from pydantic<3,>=2->ydata-profiling) (2.20.1)
 Requirement already satisfied: typing-extensions>=4.6.1 in c:\users\yaswa\anaconda3\lib\site-packages (from pydantic<3,>=2->ydata-profiling) (4.15.0)
 Requirement already satisfied: charset-normalizer<4,>=2 in c:\users\yaswa\anaconda3\lib\site-packages (from requests<3,>=2.32.0->ydata-profiling) (3.3.2)
 Requirement already satisfied: idna<4,>=2.5 in c:\users\yaswa\anaconda3\lib\site-packages (from requests<3,>=2.32.0->ydata-profiling) (3.7)
 Requirement already satisfied: urllib3<3,>=1.21.1 in c:\users\yaswa\anaconda3\lib\site-packages (from requests<3,>=2.32.0->ydata-profiling) (2.2.3)
 Requirement already satisfied: certifi>=2017.4.17 in c:\users\yaswa\anaconda3\lib\site-packages (from requests<3,>=2.32.0->ydata-profiling) (2024.8.30)
 Requirement already satisfied: patsy>=0.5.6 in c:\users\yaswa\anaconda3\lib\site-packages (from statsmodels<1,>=0.13.2->ydata-profiling) (0.5.6)
 Requirement already satisfied: colorama in c:\users\yaswa\anaconda3\lib\site-packages (from tqdm<5,>=4.66.3->ydata-profiling) (0.4.6)
 Requirement already satisfied: attrs>=19.3.0 in c:\users\yaswa\anaconda3\lib\site-packages (from visions<0.8.2,>=0.7.5->visions[type_image_path]<0.8.2,>=0.7.5->ydata-profiling) (23.1.0)
 Requirement already satisfied: networkx>=2.4 in c:\users\yaswa\anaconda3\lib\site-packages (from visions<0.8.2,>=0.7.5->visions[type_image_path]<0.8.2,>=0.7.5->ydata-profiling) (3.3)
 Requirement already satisfied: puremagic in c:\users\yaswa\anaconda3\lib\site-packages (from visions<0.8.2,>=0.7.5->visions[type_image_path]<0.8.2,>=0.7.5->ydata-profiling) (2.2.0)
 Requirement already satisfied: six in c:\users\yaswa\anaconda3\lib\site-packages (from patsy>=0.5.6->statsmodels<1,>=0.13.2->ydata-profiling) (1.16.0)

```
In [6]: import pandas as pd
        from ydata_profiling import ProfileReport

        # Load dataset
        df = pd.read_csv(r"C:\Users\yaswa\Desktop\Transaction.csv")

        # Display first 5 records
        print(df.head())
```

	Transaction_id	Customer_id	Product_id	Quality	Discount	Payment_mode	\
0	T001	C001	P001	2	5	UPI	
1	T002	C002	P002	3	10	Cash	
2	T003	C003	P003	1	0	Card	
3	T004	C004	P004	2	15	UPI	
4	T005	C005	P005	1	5	Cash	

	Sale_data
0	01-07-2026
1	02-07-2026
2	03-07-2026
3	04-07-2026
4	05-07-2026

```
In [7]: #1. Generate automated profiling report
# Generate profiling report
profile = ProfileReport(
    df,
    title="Transaction Dataset Profiling Report",
    explorative=True
)

# Display report
profile
```

Summarize dataset: 0%| | 0/5 [00:00<?, ?it/s]

0%|

| 0/7 [00:00<?, ?it/s]

100%|

| 7/7 [00:00<00:00, 26.73it/s]

Generate report structure: 0%| | 0/1 [00:00<?, ?it/s]

Render HTML: 0%| | 0/1 [00:00<?, ?it/s]

Transaction Dataset Profiling Report

Overview

Overview

Alerts 3

Reproduction

Dataset statistics

Number of variables	7
Number of observations	25
Missing cells	0
Missing cells (%)	0.0%
Duplicate rows	0
Duplicate rows (%)	0.0%
Total size in memory	7.1 KiB
Average record size in memory	291.9 B

Variable types

Text	3
Categorical	3
DateTime	1

Out[7]:

```
In [8]: #2. Identify numerical and categorical attributes
# Numerical columns
numerical_columns = df.select_dtypes(
    include=["int64", "float64"]
).columns.tolist()

print("Numerical Attributes:")
print(numerical_columns)
```

```
# Categorical columns
categorical_columns = df.select_dtypes(
    include=["object", "category"]
).columns.tolist()

print("\nCategorical Attributes:")
print(categorical_columns)
```

Numerical Attributes:
['Quality', 'Discount']

Categorical Attributes:
['Transaction_id', 'Customer_id', 'Product_id', 'Payment_mode', 'Sale_data']

```
In [9]: #3. Identify missing values
print("Missing Values:")
print(df.isnull().sum())
```

Missing Values:

Transaction_id	0
Customer_id	0
Product_id	0
Quality	0
Discount	0
Payment_mode	0
Sale_data	0

dtype: int64

```
In [10]: #4. Identify duplicate records
duplicate_count = df.duplicated().sum()

print("Number of Duplicate Records:")
print(duplicate_count)
```

Number of Duplicate Records:
0

```
In [11]: #5. Identify unique values in categorical columns
for col in categorical_columns:
    print("\nColumn:", col)
    print("Number of Unique Values:", df[col].nunique())
    print("Unique Values:")
    print(df[col].unique())
```

Column: Transaction_id
 Number of Unique Values: 25
 Unique Values:
 ['T001' 'T002' 'T003' 'T004' 'T005' 'T006' 'T007' 'T008' 'T009' 'T010'
 'T011' 'T012' 'T013' 'T014' 'T015' 'T016' 'T017' 'T018' 'T019' 'T020'
 'T021' 'T022' 'T023' 'T024' 'T025']

Column: Customer_id
 Number of Unique Values: 25
 Unique Values:
 ['C001' 'C002' 'C003' 'C004' 'C005' 'C006' 'C007' 'C008' 'C009' 'C010'
 'C011' 'C012' 'C013' 'C014' 'C015' 'C016' 'C017' 'C018' 'C019' 'C020'
 'C021' 'C022' 'C023' 'C024' 'C025']

Column: Product_id
 Number of Unique Values: 24
 Unique Values:
 ['P001' 'P002' 'P003' 'P004' 'P005' 'P006' 'P007' 'P008' 'P009' 'P010'
 'P011' 'P012' 'P013' 'P014' 'P015' 'P016' 'P017' 'P018' 'P019' 'P020'
 'P021' 'P022' 'P023' 'P024']

Column: Payment_mode
 Number of Unique Values: 3
 Unique Values:
 ['UPI' 'Cash' 'Card']

Column: Sale_data
 Number of Unique Values: 25
 Unique Values:
 ['01-07-2026' '02-07-2026' '03-07-2026' '04-07-2026' '05-07-2026'
 '06-07-2026' '07-07-2026' '08-07-2026' '09-07-2026' '10-07-2026'
 '11-07-2026' '12-07-2026' '13-07-2026' '14-07-2026' '15-07-2026'
 '16-07-2026' '17-07-2026' '18-07-2026' '19-07-2026' '20-07-2026'
 '21-07-2026' '22-07-2026' '23-07-2026' '24-07-2026' '25-07-2026']

```
In [12]: #6. Analyze distributions of numerical variables
print("Numerical Variable Statistics:")
print(df[numerical_columns].describe())
```

```
Numerical Variable Statistics:
      Quality  Discount
count  25.00000  25.00000
mean    2.36000    7.20000
std     1.18603    5.016639
min     1.00000    0.00000
25%     2.00000    5.00000
50%     2.00000    5.00000
75%     3.00000   10.00000
max     5.00000   15.00000
```

```
In [14]: import matplotlib.pyplot as plt

for col in numerical_columns:
    plt.figure(figsize=(8, 5))

    plt.hist(
        df[col].dropna(),
        bins=30
    )
```

```
plt.title("Distribution of " + col)
plt.xlabel(col)
plt.ylabel("Frequency")

plt.show()
```

In [16]: %matplotlib inline

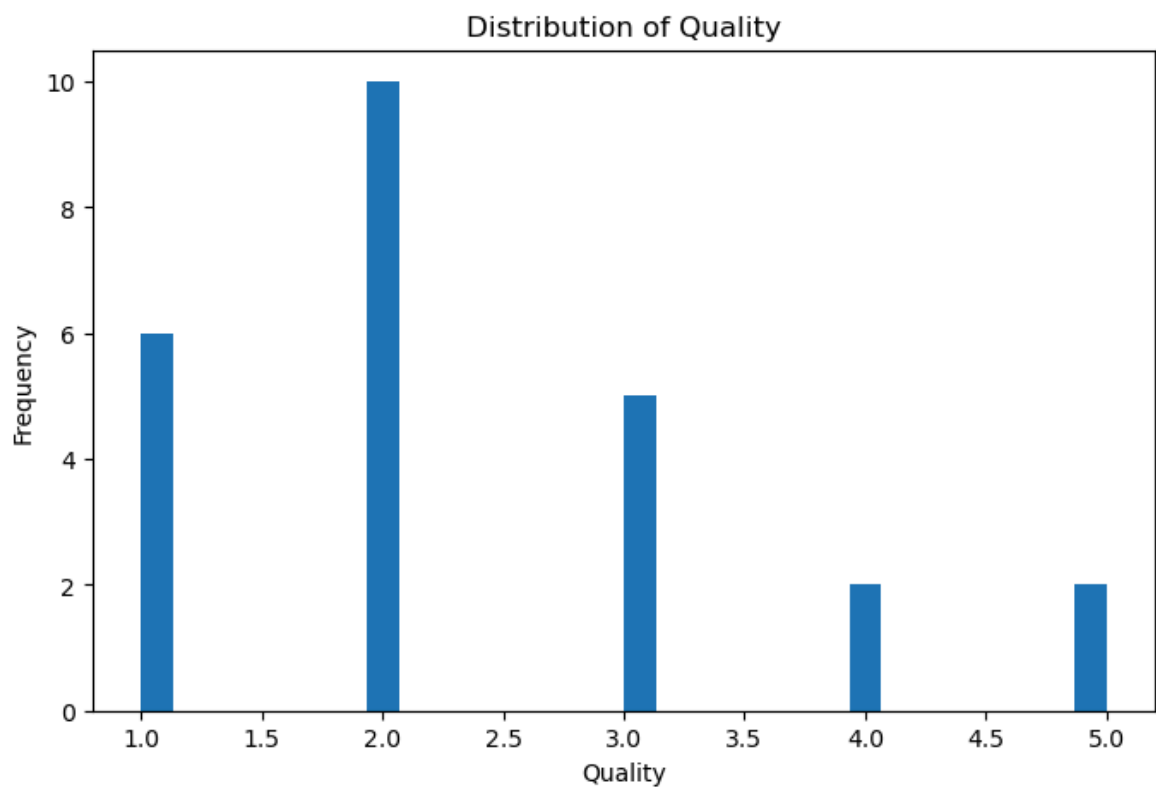
```
import matplotlib.pyplot as plt

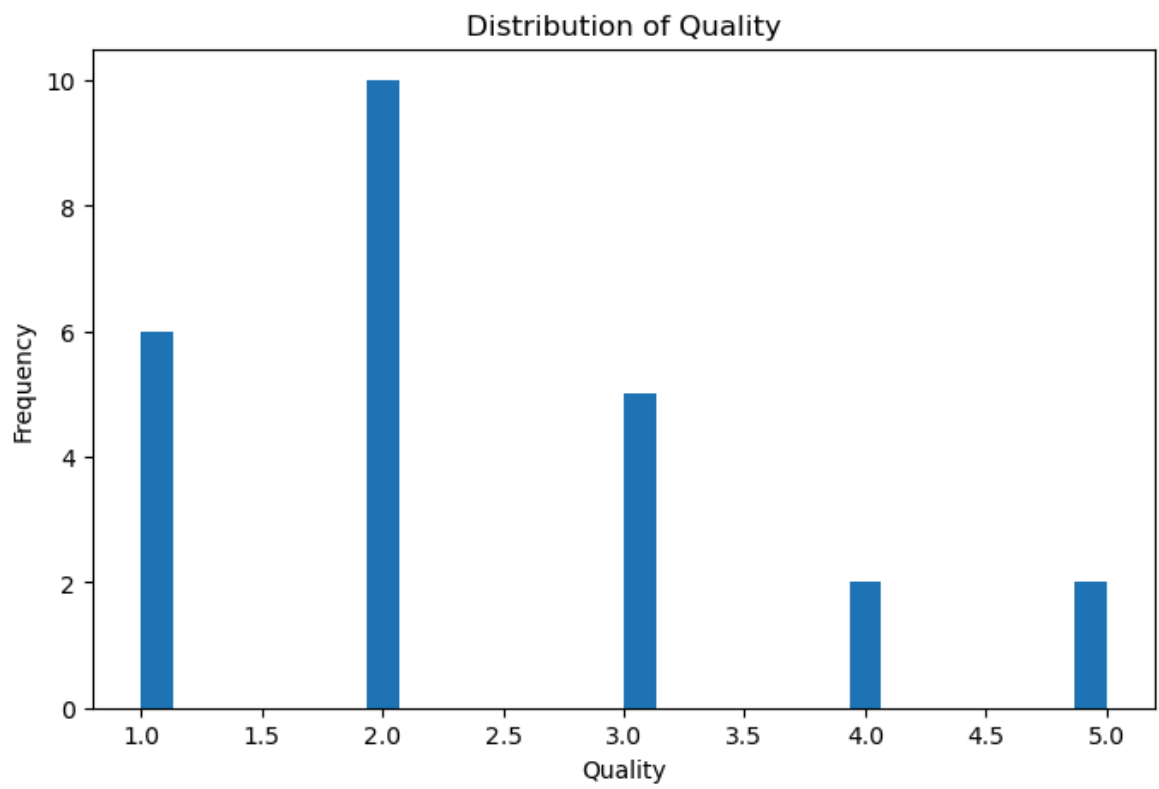
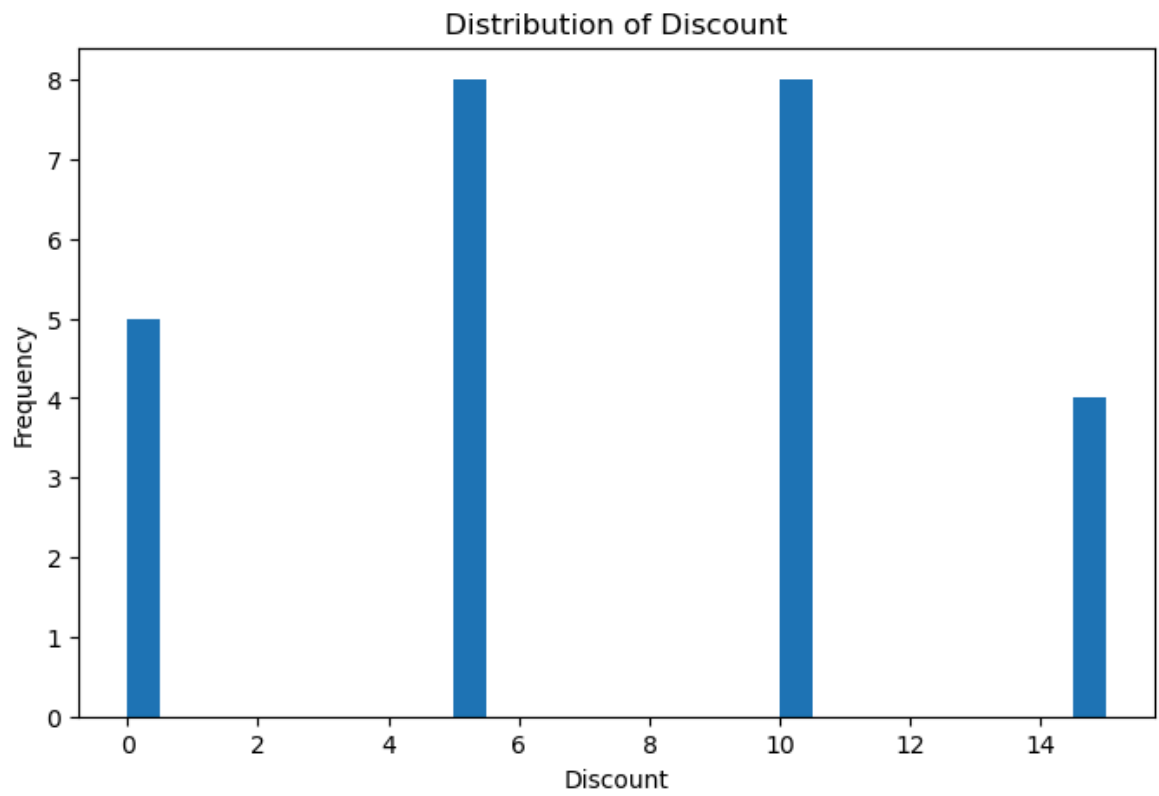
for col in numerical_columns:
    plt.figure(figsize=(8, 5))

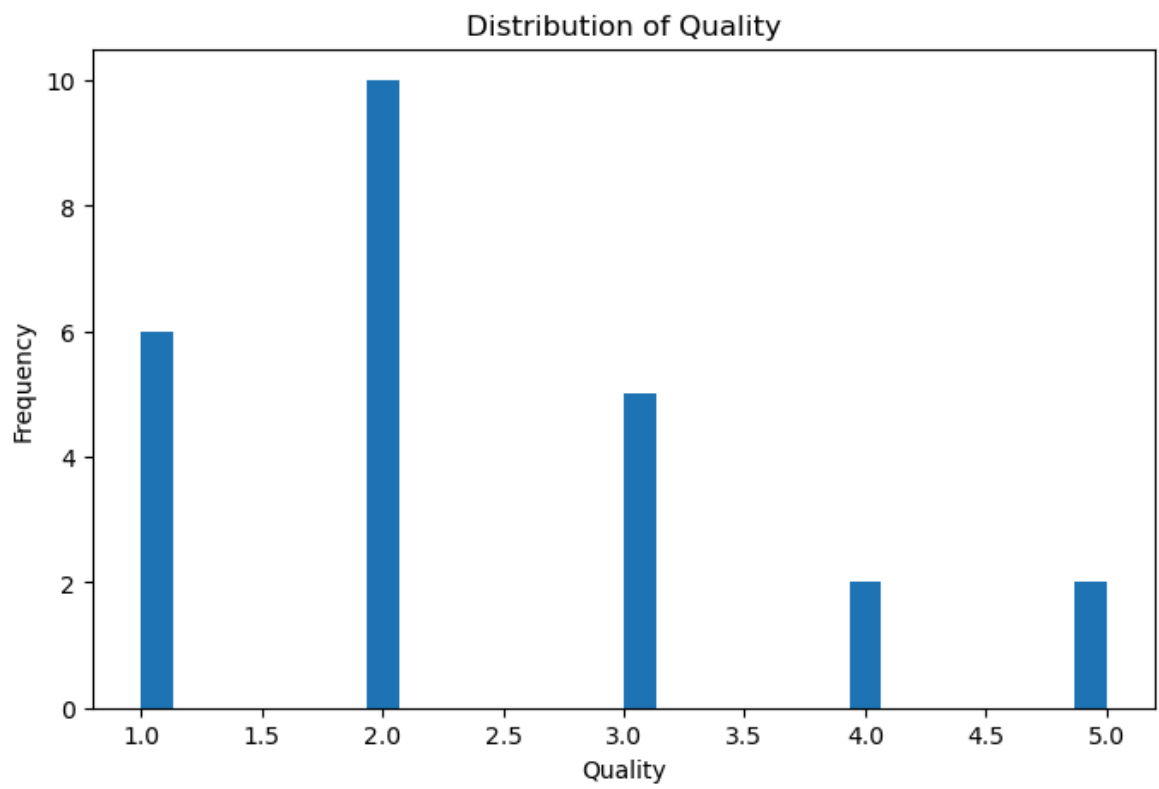
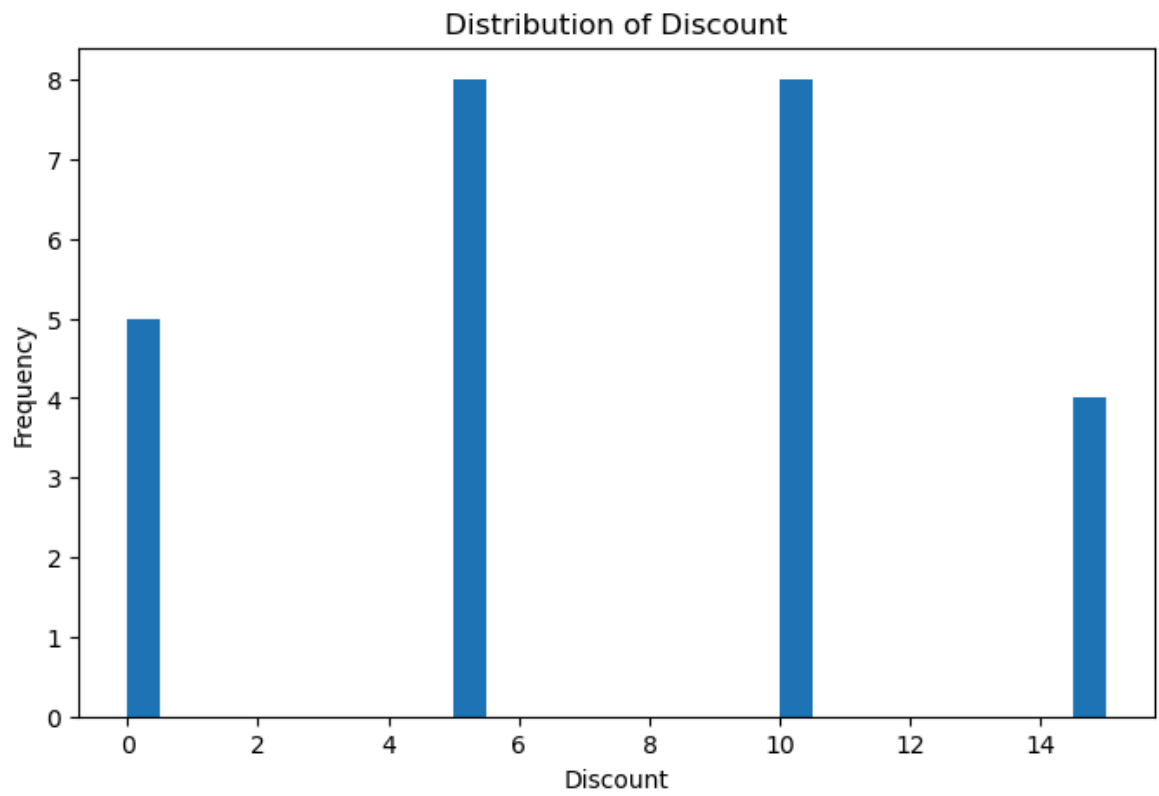
    plt.hist(
        df[col].dropna(),
        bins=30
    )

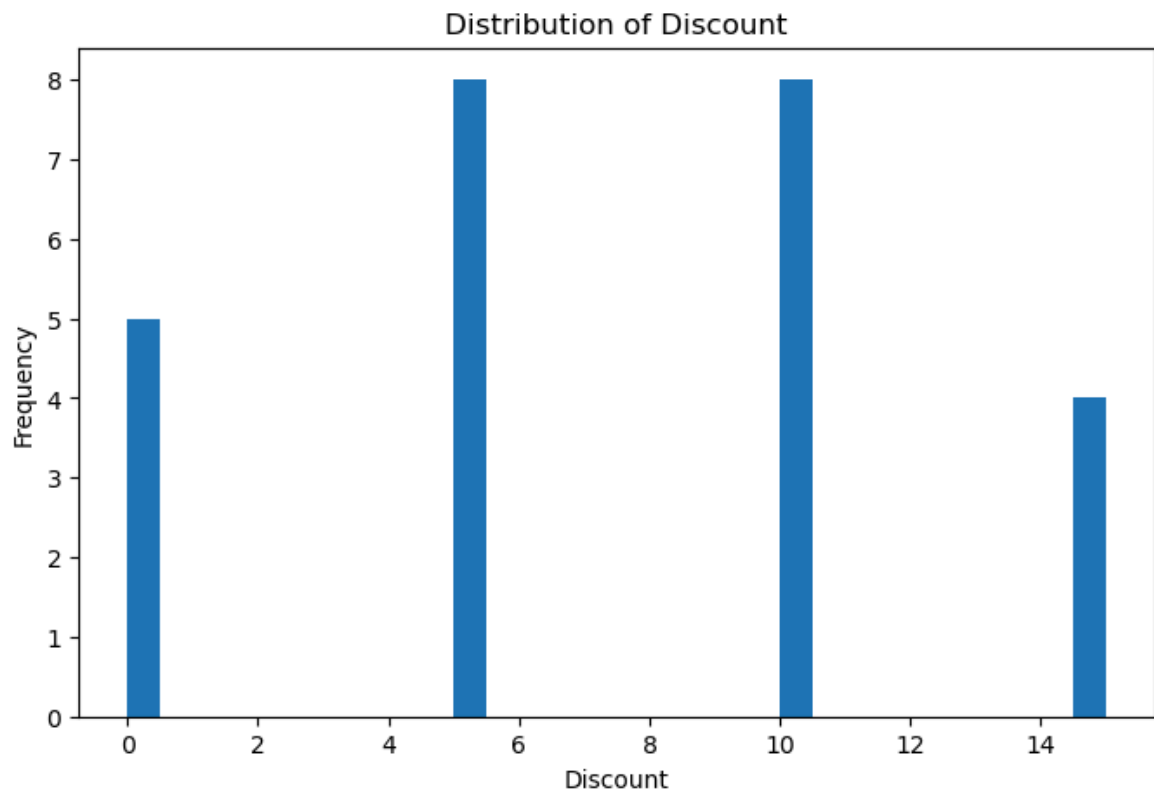
    plt.title("Distribution of " + col)
    plt.xlabel(col)
    plt.ylabel("Frequency")

    plt.show()
```









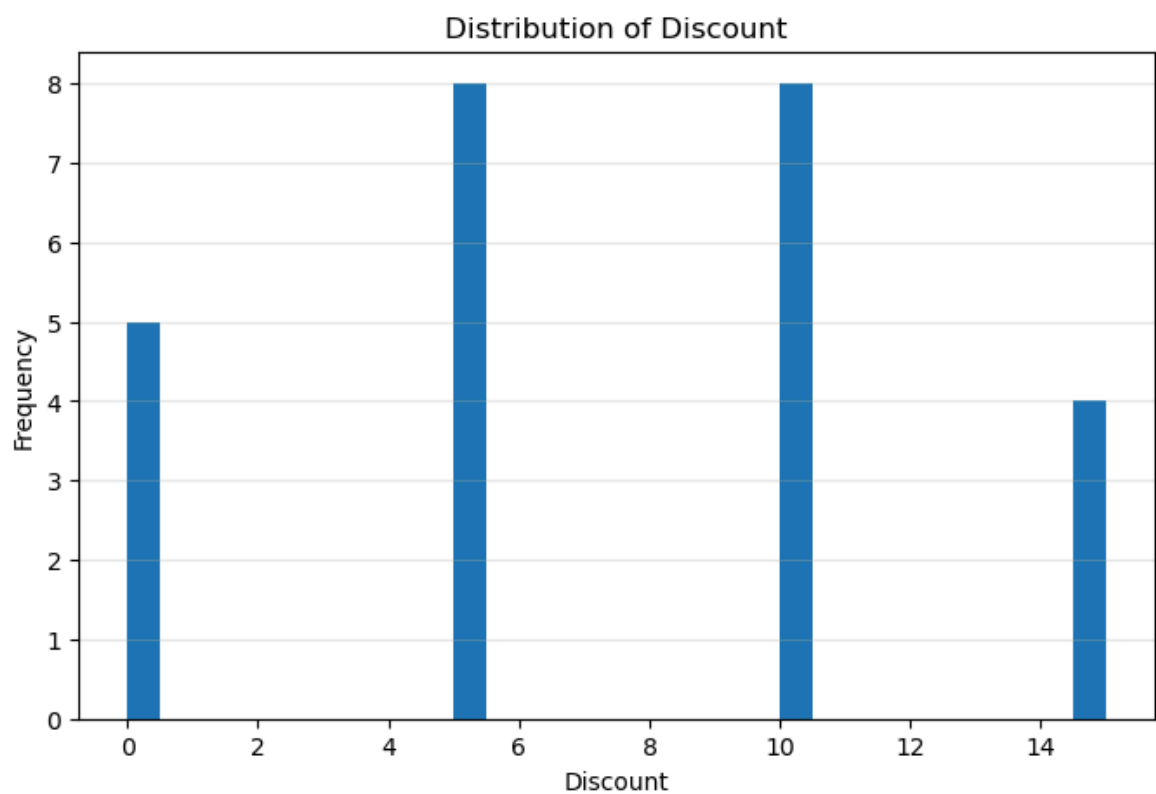
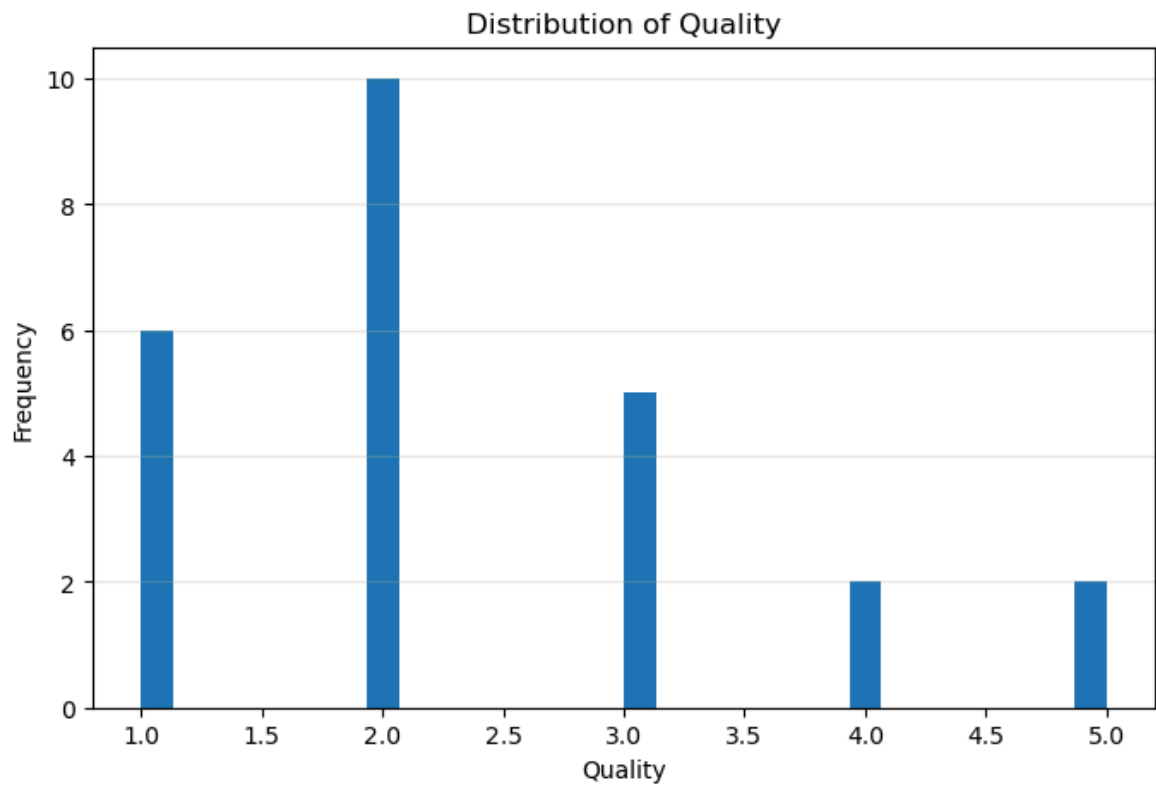
```
In [17]: import matplotlib.pyplot as plt

for col in numerical_columns:
    plt.figure(figsize=(8, 5))

    plt.hist(df[col].dropna(), bins=30)

    plt.title(f"Distribution of {col}")
    plt.xlabel(col)
    plt.ylabel("Frequency")
    plt.grid(axis="y", alpha=0.3)

    plt.show()
    plt.close()
```



```
In [18]: #7. Identify possible outliers
for col in numerical_columns:

    Q1 = df[col].quantile(0.25)
    Q3 = df[col].quantile(0.75)

    IQR = Q3 - Q1

    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR
```



```
outliers = df[
    (df[col] < lower_bound) |
    (df[col] > upper_bound)
]

print("\nColumn:", col)
print("Lower Bound:", lower_bound)
print("Upper Bound:", upper_bound)
print("Number of Outliers:", len(outliers))
```

Column: Quality
Lower Bound: 0.5
Upper Bound: 4.5
Number of Outliers: 2

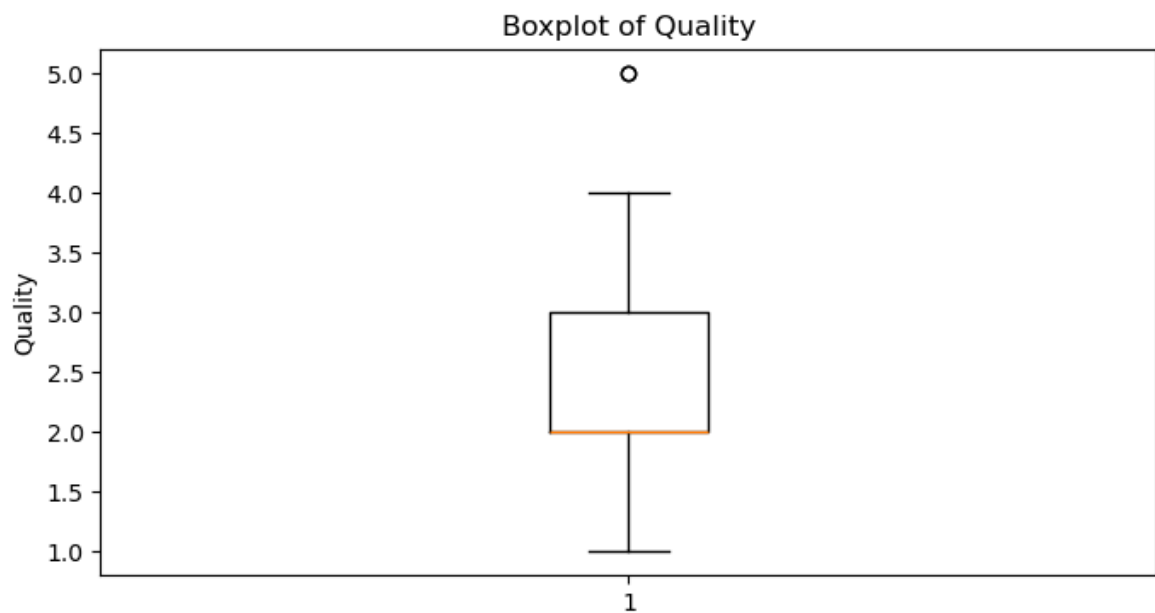
Column: Discount
Lower Bound: -2.5
Upper Bound: 17.5
Number of Outliers: 0

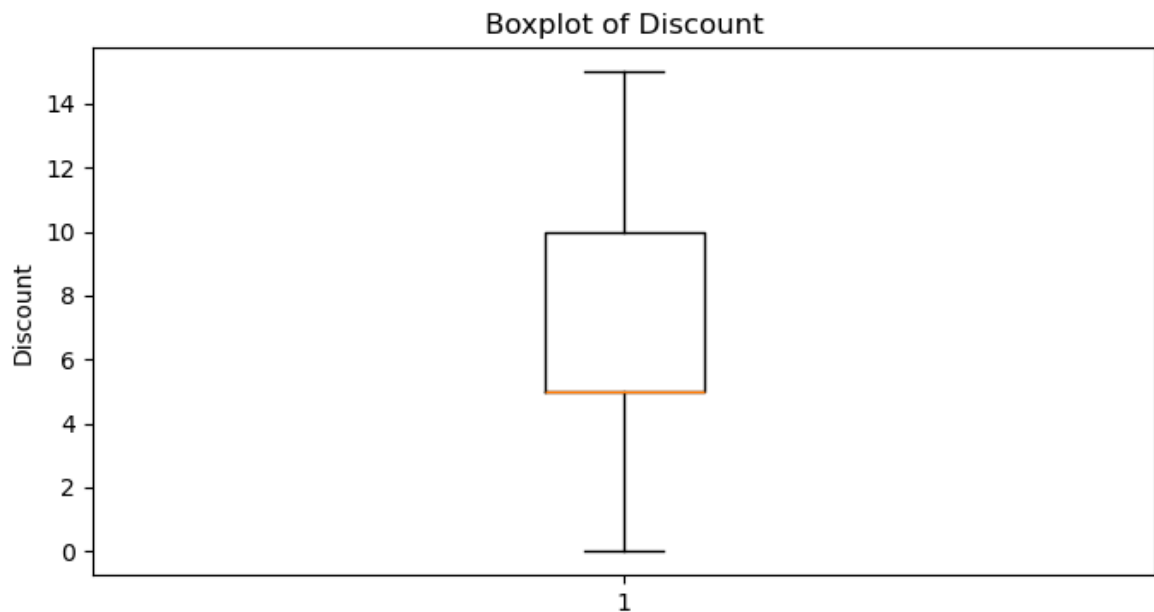
```
In [19]: for col in numerical_columns:
plt.figure(figsize=(8, 4))

plt.boxplot(df[col].dropna())

plt.title("Boxplot of " + col)
plt.ylabel(col)

plt.show()
```





```
In [20]: #8. Examine correlations between numerical variables
correlation_matrix = df[numerical_columns].corr()

print("Correlation Matrix:")
print(correlation_matrix)
```

Correlation Matrix:

	Quality	Discount
Quality	1.000000	0.456592
Discount	0.456592	1.000000

```
In [21]: plt.figure(figsize=(10, 7))

plt.imshow(
    correlation_matrix,
    cmap="coolwarm",
    interpolation="nearest"
)

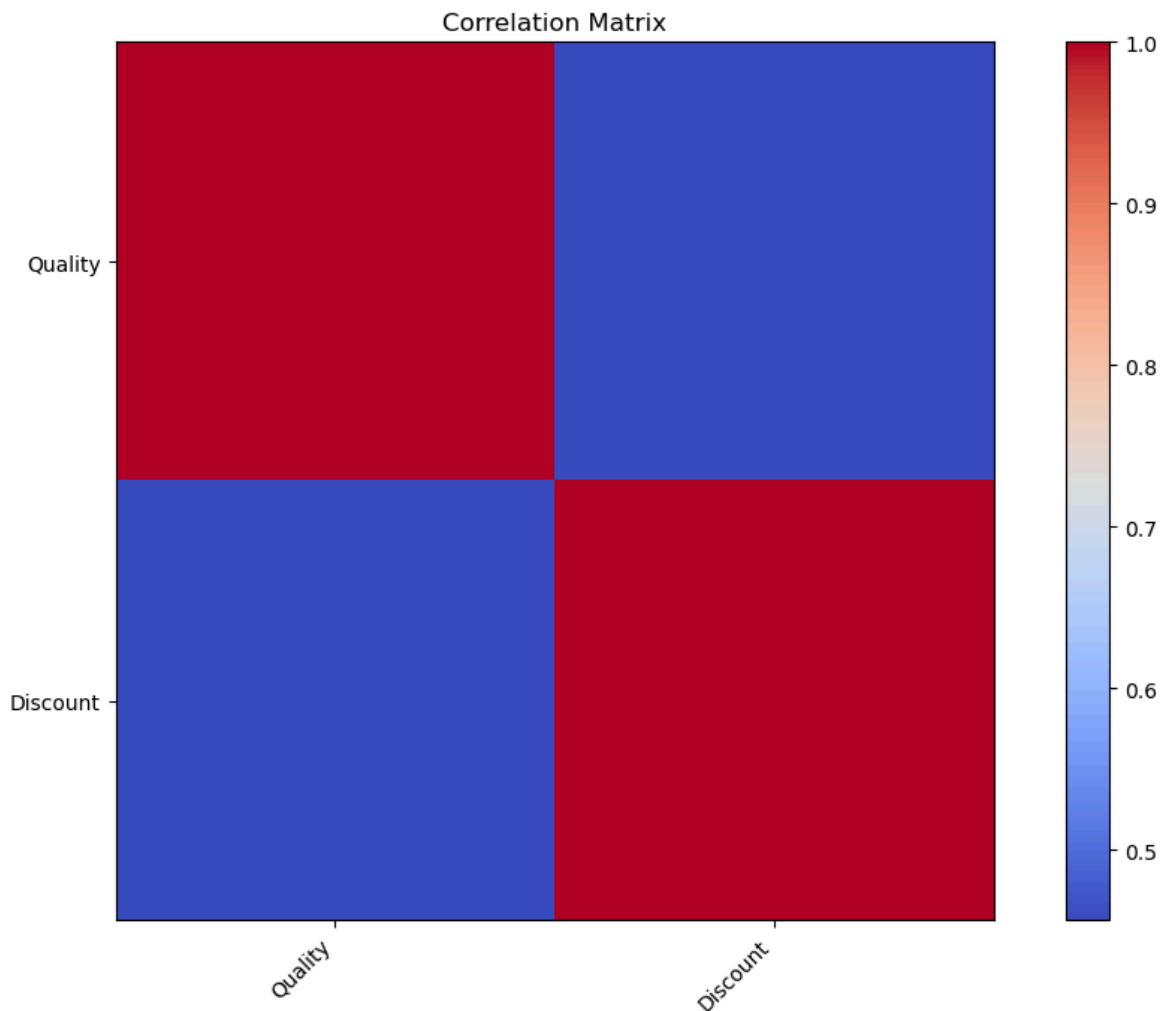
plt.colorbar()

plt.xticks(
    range(len(correlation_matrix.columns)),
    correlation_matrix.columns,
    rotation=45,
    ha="right"
)

plt.yticks(
    range(len(correlation_matrix.columns)),
    correlation_matrix.columns
)

plt.title("Correlation Matrix")

plt.tight_layout()
plt.show()
```



```
In [22]: #9. Save profiling report as HTML
profile.to_file(
    "Transaction_profiling_report.html"
)

print("Profiling report saved successfully!")
```

Export report to file: 0%| | 0/1 [00:00<?, ?it/s]
 Profiling report saved successfully!

```
In [23]: import pandas as pd
from ydata_profiling import ProfileReport

# Load dataset
df = pd.read_csv(r"C:\Users\yaswa\Desktop\Transaction.csv")

# Display first 5 records
print(df.head())

# 1. Generate profiling report
profile = ProfileReport(
    df,
    title="Transaction Dataset Profiling Report",
    explorative=True
)

profile
```

	Transaction_id	Customer_id	Product_id	Quality	Discount	Payment_mode	\
0	T001	C001	P001	2	5	UPI	
1	T002	C002	P002	3	10	Cash	
2	T003	C003	P003	1	0	Card	
3	T004	C004	P004	2	15	UPI	
4	T005	C005	P005	1	5	Cash	

Sale_data
0 01-07-2026
1 02-07-2026
2 03-07-2026
3 04-07-2026
4 05-07-2026

Summarize dataset: 0%| | 0/5 [00:00<?, ?it/s]



Generate report structure: 0%| | 0/1 [00:00<?, ?it/s]

Render HTML: 0%| | 0/1 [00:00<?, ?it/s]

Transaction Dataset Profiling Report



Overview

- Overview
- Alerts 3
- Reproduction

Dataset statistics

Number of variables	7
Number of observations	25
Missing cells	0
Missing cells (%)	0.0%
Duplicate rows	0
Duplicate rows (%)	0.0%
Total size in memory	7.1 KiB
Average record size in memory	291.9 B

Variable types

Text	3
Categorical	3
DateTime	1



Out[23]:

```
In [24]: !pip install pandera
```

Collecting pandera

Downloading pandera-0.32.1-py3-none-any.whl.metadata (10 kB)

Requirement already satisfied: packaging>=20.0 in c:\users\yaswa\anaconda3\lib\site-packages (from pandera) (24.1)

Requirement already satisfied: pydantic in c:\users\yaswa\anaconda3\lib\site-packages (from pandera) (2.8.2)

Requirement already satisfied: typeguard in c:\users\yaswa\anaconda3\lib\site-packages (from pandera) (4.6.0)

Requirement already satisfied: typing_extensions in c:\users\yaswa\anaconda3\lib\site-packages (from pandera) (4.15.0)

Collecting typing_inspect>=0.6.0 (from pandera)

Downloading typing_inspect-0.9.0-py3-none-any.whl.metadata (1.5 kB)

Requirement already satisfied: mypy_extensions>=0.3.0 in c:\users\yaswa\anaconda3\lib\site-packages (from typing_inspect>=0.6.0->pandera) (1.0.0)

Requirement already satisfied: annotated_types>=0.4.0 in c:\users\yaswa\anaconda3\lib\site-packages (from pydantic->pandera) (0.6.0)

Requirement already satisfied: pydantic-core==2.20.1 in c:\users\yaswa\anaconda3\lib\site-packages (from pydantic->pandera) (2.20.1)

Downloading pandera-0.32.1-py3-none-any.whl (447 kB)

Downloading typing_inspect-0.9.0-py3-none-any.whl (8.8 kB)

Installing collected packages: typing_inspect, pandera

Successfully installed pandera-0.32.1 typing_inspect-0.9.0

```
In [28]: import pandera as pa
        from pandera import Column, DataFrameSchema, Check
```

```
In [29]: import pandera.pandas as pa
        from pandera.pandas import Column, DataFrameSchema, Check
```

```
In [34]: # Create Pandera validation schema
        schema = DataFrameSchema({

            # Quantity Sold
            "Quantity_Sold": Column(
                float,
                nullable=False,
                checks=Check.ge(0)
            ),

            # Unit Cost
            "Unit_Cost": Column(
                float,
                nullable=False,
                checks=Check.ge(0)
            ),

            # Unit Price
            "Unit_Price": Column(
                float,
                nullable=False,
                checks=Check.ge(0)
            ),

            # Discount
            "Discount": Column(
                float,
                nullable=False,
                checks=Check.in_range(0, 1)
            ),
```

```

# Sales Amount
"Sales_Amount": Column(
    float,
    nullable=False,
    checks=Check.ge(0)
),
strict=False
)

```

```

In [40]: print(df.columns.tolist())
print(df.head())

```

```

['Transaction_id', 'Customer_id', 'Product_id', 'Quality', 'Discount', 'Payment_m
ode', 'Sale_data']

```

	Transaction_id	Customer_id	Product_id	Quality	Discount	Payment_mode \
0	T001	C001	P001	2	5	UPI
1	T002	C002	P002	3	10	Cash
2	T003	C003	P003	1	0	Card
3	T004	C004	P004	2	15	UPI
4	T005	C005	P005	1	5	Cash

	Sale_data
0	01-07-2026
1	02-07-2026
2	03-07-2026
3	04-07-2026
4	05-07-2026

```

In [45]: %whos

```

Variable	Type	Data/Info
Check	MetaCheck	<class 'pandera.api.checks.Check'>
Column	ABCMeta	<class 'pandera.api.pandas.components.Co lumn'>
DataFrameSchema	ABCMeta	<class 'pandera.api.panda<...>ntainer.Da taFrameSchema'>
IQR	float64	5.0
ProfileReport	type	<class 'ydata_profiling.p<...>le_report. ProfileReport'>
Q1	float64	5.0
Q3	float64	10.0
categorical_columns	list	n=5
col	str	Discount
correlation_matrix	DataFrame	Quality Disco<...>count 0.4 56592 1.000000
df	DataFrame	Transaction_id Custome<...>-2026 \n2 4 25-07-2026
duplicate_count	int64	0
lower_bound	float64	-2.5
numerical_columns	list	n=2
outliers	DataFrame	Empty DataFrame\nColumns:<...>de, Sale_d ata]\nIndex: []
pa	module	<module 'pandera.pandas' <...>ges\\pande ra\\pandas.py'>
pd	module	<module 'pandas' from 'C:<...>es\\pandas __init__.py'>
plt	module	<module 'matplotlib.pyplot<...>\\matplotl ib\\pyplot.py'>
profile	ProfileReport	
schema	DataFrameSchema	<Schema DataFrameSchema(\\<...>missing_co lums=False\n)>
upper_bound	float64	17.5

```
In [46]: print(df.columns.tolist())
print(df.head())
print(df.dtypes)
```



```
['Transaction_id', 'Customer_id', 'Product_id', 'Quality', 'Discount', 'Payment_m
ode', 'Sale_data']
```

	Transaction_id	Customer_id	Product_id	Quality	Discount	Payment_mode	\
0	T001	C001	P001	2	5	UPI	
1	T002	C002	P002	3	10	Cash	
2	T003	C003	P003	1	0	Card	
3	T004	C004	P004	2	15	UPI	
4	T005	C005	P005	1	5	Cash	

```
Sale_data
```

```
0 01-07-2026
1 02-07-2026
2 03-07-2026
3 04-07-2026
4 05-07-2026
```

```
Transaction_id    object
Customer_id       object
Product_id        object
Quality           int64
Discount          int64
Payment_mode      object
Sale_data         object
dtype: object
```

```
In [48]: df = pd.read_csv(r"C:\Users\yaswa\Desktop\Transaction.csv")

print(df.columns.tolist())
```

```
['Transaction_id', 'Customer_id', 'Product_id', 'Quality', 'Discount', 'Payment_m
ode', 'Sale_data']
```

```
In [49]: required_columns = [
    "Quantity_Sold",
    "Unit_Cost",
    "Unit_Price",
    "Discount",
    "Sales_Amount"
]

print([col for col in required_columns if col not in df.columns])
```

```
['Quantity_Sold', 'Unit_Cost', 'Unit_Price', 'Sales_Amount']
```

```
In [52]: print(schema.columns.keys())
```

```
dict_keys(['Quantity_Sold', 'Unit_Cost', 'Unit_Price', 'Discount', 'Sales_Amount'])
```

```
In [54]: print(schema.columns.keys())
```

```
dict_keys(['Transaction_id', 'Customer_id', 'Product_id', 'Quality', 'Discount',
'Payment_mode', 'Sale_data'])
```

```
In [56]: print(df.dtypes)
```

```
Transaction_id    object
Customer_id       object
Product_id        object
Quality           int64
Discount          int64
Payment_mode      object
Sale_data         object
dtype: object
```

```
In [57]: import pandera.pandas as pa
from pandera.pandas import DataFrameSchema, Column, Check

schema = DataFrameSchema({

    "Transaction_id": Column(
        str,
        nullable=False
    ),

    "Customer_id": Column(
        str,
        nullable=False
    ),

    "Product_id": Column(
        str,
        nullable=False
    ),

    "Quality": Column(
        float,
        nullable=False,
        checks=Check.ge(0)
    ),

    "Discount": Column(
        float,
        nullable=False,
        checks=Check.in_range(0, 100)
    ),

    "Payment_mode": Column(
        str,
        nullable=False
    ),

    "Sale_data": Column(
        str,
        nullable=False
    )

}, strict=False)
```

```
In [59]: try:
    validated_df = schema.validate(df, lazy=True)
    print("Data validation successful!")
except pa.errors.SchemaErrors as e:
    print(e.failure_cases)
```

	column	failure_case	index	schema_context	check	check_number
0	Quality	int64	None	Column	dtype('float64')	None
1	Discount	int64	None	Column	dtype('float64')	None

```
In [60]: #Stage 4 - Data Engineering
sales_processed = df.copy()
```

```
In [61]: print("Duplicates before:", sales_processed.duplicated().sum())

sales_processed = sales_processed.drop_duplicates()

print("Duplicates after:", sales_processed.duplicated().sum())
```

Duplicates before: 0

Duplicates after: 0

```
In [62]: print(sales_processed.isnull().sum())
```

```
Transaction_id    0
Customer_id       0
Product_id        0
Quality           0
Discount          0
Payment_mode      0
Sale_data         0
dtype: int64
```

```
In [63]: numeric_cols = sales_processed.select_dtypes(include="number").columns

for col in numeric_cols:
    sales_processed[col] = sales_processed[col].fillna(
        sales_processed[col].median()
    )
```

```
In [64]: categorical_cols = sales_processed.select_dtypes(include="object").columns

for col in categorical_cols:
    if sales_processed[col].isnull().sum() > 0:
        sales_processed[col] = sales_processed[col].fillna(
            sales_processed[col].mode()[0]
        )
```

```
In [65]: #4. Correct inappropriate data types
sales_processed["Quality"] = pd.to_numeric(
    sales_processed["Quality"], errors="coerce"
)

sales_processed["Discount"] = pd.to_numeric(
    sales_processed["Discount"], errors="coerce"
)
```

```
In [66]: sales_processed["Transaction_id"] = sales_processed["Transaction_id"].astype(str)
sales_processed["Customer_id"] = sales_processed["Customer_id"].astype(str)
sales_processed["Product_id"] = sales_processed["Product_id"].astype(str)
```

```
In [67]: sales_processed["Payment_mode"] = (
    sales_processed["Payment_mode"]
    .astype(str)
    .str.strip()
```

```
.str.lower()  
)
```

```
In [68]: sales_processed["Payment_mode"] = sales_processed["Payment_mode"].replace({  
        "cash": "Cash",  
        "card": "Card",  
        "credit card": "Card",  
        "debit card": "Card",  
        "upi": "UPI",  
        "online": "UPI"  
    })
```

```
In [69]: sales_processed["Sale_data"] = pd.to_datetime(  
        sales_processed["Sale_data"],  
        errors="coerce",  
        dayfirst=True  
    )
```

```
In [70]: print("Invalid dates:", sales_processed["Sale_data"].isna().sum())
```

Invalid dates: 0

```
In [71]: #7. Handle invalid numerical values  
sales_processed.loc[  
    sales_processed["Quality"] < 0, "Quality"  
] = sales_processed["Quality"].median()  
  
sales_processed.loc[  
    (sales_processed["Discount"] < 0) |  
    (sales_processed["Discount"] > 100),  
    "Discount"  
] = sales_processed["Discount"].median()
```

```
In [72]: #8. Save the processed dataset  
sales_processed.to_csv("sales_processed.csv", index=False)  
  
print("Processed dataset saved successfully!")
```

Processed dataset saved successfully!

```
In [73]: print(sales_processed.head())  
print(sales_processed.dtypes)  
print(sales_processed.shape)
```

	Transaction_id	Customer_id	Product_id	Quality	Discount	Payment_mode	\
0	T001	C001	P001	2	5	UPI	
1	T002	C002	P002	3	10	Cash	
2	T003	C003	P003	1	0	Card	
3	T004	C004	P004	2	15	UPI	
4	T005	C005	P005	1	5	Cash	

```

Sale_data
0 2026-07-01
1 2026-07-02
2 2026-07-03
3 2026-07-04
4 2026-07-05
Transaction_id      object
Customer_id         object
Product_id          object
Quality             int64
Discount            int64
Payment_mode        object
Sale_data           datetime64[ns]
dtype: object
(25, 7)

```

```

In [74]: import pandas as pd

# 1. Create a copy of raw data
sales_processed = df.copy()

# 2. Remove duplicates
sales_processed = sales_processed.drop_duplicates()

# 3. Correct numerical data types
sales_processed["Quality"] = pd.to_numeric(
    sales_processed["Quality"], errors="coerce"
)

sales_processed["Discount"] = pd.to_numeric(
    sales_processed["Discount"], errors="coerce"
)

# 4. Handle missing numerical values
for col in ["Quality", "Discount"]:
    sales_processed[col] = sales_processed[col].fillna(
        sales_processed[col].median()
    )

# 5. Handle missing categorical values
categorical_cols = ["Payment_mode"]

for col in categorical_cols:
    sales_processed[col] = sales_processed[col].fillna(
        sales_processed[col].mode()[0]
    )

# 6. Standardize categorical values
sales_processed["Payment_mode"] = (
    sales_processed["Payment_mode"]
    .astype(str)
    .str.strip()
    .str.lower()
)

```

```

)

sales_processed["Payment_mode"] = sales_processed["Payment_mode"].replace({
    "cash": "Cash",
    "card": "Card",
    "credit card": "Card",
    "debit card": "Card",
    "upi": "UPI",
    "online": "UPI"
})

# 7. Convert date to datetime
sales_processed["Sale_data"] = pd.to_datetime(
    sales_processed["Sale_data"],
    errors="coerce",
    dayfirst=True
)

# 8. Handle invalid numerical values
sales_processed.loc[
    sales_processed["Quality"] < 0, "Quality"
] = sales_processed["Quality"].median()

sales_processed.loc[
    (sales_processed["Discount"] < 0) |
    (sales_processed["Discount"] > 100),
    "Discount"
] = sales_processed["Discount"].median()

# 9. Handle invalid dates
sales_processed = sales_processed.dropna(subset=["Sale_data"])

# 10. Save processed dataset
sales_processed.to_csv("sales_processed.csv", index=False)

print("Data engineering completed successfully!")
print("Final shape:", sales_processed.shape)
print("Saved as: sales_processed.csv")

```

Data engineering completed successfully!
 Final shape: (25, 7)
 Saved as: sales_processed.csv

```

In [13]: import pandas as pd
import os
import numpy as np

def ingest_data():

    df = pd.read_csv(
        r"C:\Users\yaswa\Desktop\Transaction.csv"
    )

    print("1. Data ingestion completed.")

    return df

def profile_data(df):

```

```
print("\n2. Data profiling")

print("Shape:", df.shape)

print("\nMissing values:")
print(df.isnull().sum())

print(
    "\nDuplicate records:",
    df.duplicated().sum()
)

return df

def validate_data(df):

    print("\n3. Data validation")

    if "Quantity_Sold" in df.columns:

        print(
            "Negative Quantity:",
            (df["Quantity_Sold"] < 0).sum()
        )

    if "Unit_Cost" in df.columns:

        print(
            "Negative Unit Cost:",
            (df["Unit_Cost"] < 0).sum()
        )

    if "Unit_Price" in df.columns:

        print(
            "Negative Unit Price:",
            (df["Unit_Price"] < 0).sum()
        )

    return df

def clean_data(df):

    print("\n4. Cleaning data")

    # Remove duplicates
    df = df.drop_duplicates()

    # Numerical columns
    numerical_columns = df.select_dtypes(
        include=["int64", "float64"]
    ).columns

    # Fill numerical missing values
    for column in numerical_columns:

        df[column] = df[column].fillna(
```

```
        df[column].median()
    )

    # Categorical columns
    categorical_columns = df.select_dtypes(
        include=["object"]
    ).columns

    for column in categorical_columns:

        df[column] = df[column].fillna(
            "Unknown"
        )

    return df

def transform_data(df):

    print("\n5. Transformation")

    # Convert date
    if "Sale_Date" in df.columns:

        df["Sale_Date"] = pd.to_datetime(
            df["Sale_Date"],
            errors="coerce"
        )

    # Standardize categorical values
    categorical_columns = df.select_dtypes(
        include=["object"]
    ).columns

    for column in categorical_columns:

        df[column] = (
            df[column]
            .astype(str)
            .str.strip()
            .str.lower()
        )

    return df

def save_processed_data(df):

    print("\n6. Saving data")

    os.makedirs(
        "data/processed",
        exist_ok=True
    )

    df.to_csv(
        "data/processed/sales_processed.csv",
        index=False
    )
```



```

    print("sales_processed.csv created.")

def main_pipeline():

    df = ingest_data()

    df = profile_data(df)

    df = validate_data(df)

    df = clean_data(df)

    df = transform_data(df)

    save_processed_data(df)

    print(
        "\nComplete pipeline executed successfully."
    )

main_pipeline()

```

1. Data ingestion completed.

2. Data profiling

Shape: (25, 7)

Missing values:

Transaction_id	0
Customer_id	0
Product_id	0
Quality	0
Discount	0
Payment_mode	0
Sale_data	0
dtype:	int64

Duplicate records: 0

3. Data validation

4. Cleaning data

5. Transformation

6. Saving data

sales_processed.csv created.

Complete pipeline executed successfully.

```

In [15]: import pandas as pd

df = pd.read_csv(
    r"C:\Users\yaswa\Desktop\Transaction.csv"
)

print(df.columns.tolist())

```

```
['Transaction_id', 'Customer_id', 'Product_id', 'Quality', 'Discount', 'Payment_m  
ode', 'Sale_data']
```

```
In [16]: import pandas as pd
import os

# Load dataset
df = pd.read_csv(
    r"C:\Users\yaswa\Desktop\Transaction.csv"
)

# Remove extra spaces from column names
df.columns = df.columns.str.strip()

# Display columns
print("Available columns:")
print(df.columns.tolist())

# Required columns
required_columns = [
    "Quantity_Sold",
    "Unit_Price",
    "Unit_Cost",
    "Discount"
]

# Check missing columns
missing_columns = []

for col in required_columns:
    if col not in df.columns:
        missing_columns.append(col)

if missing_columns:
    print("\nMissing columns:")
    print(missing_columns)
    print("\nPlease check the actual column names shown above.")
else:

    # Gross Sales Value
    df["Gross_Sales_Value"] = (
        df["Quantity_Sold"] *
        df["Unit_Price"]
    )

    # Total Cost
    df["Total_Cost"] = (
        df["Quantity_Sold"] *
        df["Unit_Cost"]
    )

    # Estimated Margin
    df["Estimated_Margin"] = (
        df["Gross_Sales_Value"] -
        df["Total_Cost"]
    )

    # Discount Amount
    df["Discount_Amount"] = (
```

```

        df["Gross_Sales_Value"] *
        df["Discount"] / 100
    )

    # Date Features
    if "Sale_Date" in df.columns:

        df["Sale_Date"] = pd.to_datetime(
            df["Sale_Date"],
            errors="coerce"
        )

        df["Sale_Year"] = df["Sale_Date"].dt.year

        df["Sale_Month"] = df["Sale_Date"].dt.month

        df["Sale_Day"] = df["Sale_Date"].dt.day

        df["Day_of_Week"] = df["Sale_Date"].dt.dayofweek

    # Create output folder
    os.makedirs("data/features", exist_ok=True)

    # Save file
    df.to_csv(
        "data/features/sales_features.csv",
        index=False
    )

    print("\nFeature engineering completed successfully.")
    print(df.head())

```

Available columns:

```
['Transaction_id', 'Customer_id', 'Product_id', 'Quality', 'Discount', 'Payment_m
ode', 'Sale_data']
```

Missing columns:

```
['Quantity_Sold', 'Unit_Price', 'Unit_Cost']
```

Please check the actual column names shown above.

```

In [17]: #Stage 8 - Prepare ML-Ready Data
import pandas as pd
from sklearn.model_selection import train_test_split

```

```

In [19]: import pandas as pd

df = pd.read_csv(
    r"C:\Users\yaswa\Desktop\Sales_MLOps\data\raw\sales_data.csv"
)

print("Columns:")
print(df.columns.tolist())

print("\nShape:")
print(df.shape)

print("\nFirst 5 rows:")
print(df.head())

```

Columns:

```
['Transaction_id', 'Customer_id', 'Product_id', 'Quality', 'Discount', 'Payment_mode', 'Sale_data']
```

Shape:

```
(25, 7)
```

First 5 rows:

	Transaction_id	Customer_id	Product_id	Quality	Discount	Payment_mode	\
0	T001	C001	P001	2	5	UPI	
1	T002	C002	P002	3	10	Cash	
2	T003	C003	P003	1	0	Card	
3	T004	C004	P004	2	15	UPI	
4	T005	C005	P005	1	5	Cash	

	Sale_data
0	01-07-2026
1	02-07-2026
2	03-07-2026
3	04-07-2026
4	05-07-2026

```
In [21]: #Stage 8 - Prepare ML-Ready Data
#Program 8.1 - Load and check data
import pandas as pd

# Load feature-engineered dataset
df = pd.read_csv(
    r"C:\Users\yaswa\Desktop\Sales_MLOps\data\raw\sales_data.csv"
)

print("Dataset loaded successfully!")

print("\nFirst 5 rows:")
print(df.head())

print("\nDataset shape:")
print(df.shape)

print("\nColumns:")
print(df.columns.tolist())
```

Dataset loaded successfully!

First 5 rows:

	Transaction_id	Customer_id	Product_id	Quality	Discount	Payment_mode	\
0	T001	C001	P001	2	5	UPI	
1	T002	C002	P002	3	10	Cash	
2	T003	C003	P003	1	0	Card	
3	T004	C004	P004	2	15	UPI	
4	T005	C005	P005	1	5	Cash	

	Sale_data
0	01-07-2026
1	02-07-2026
2	03-07-2026
3	04-07-2026
4	05-07-2026

Dataset shape:
(25, 7)

Columns:
['Transaction_id', 'Customer_id', 'Product_id', 'Quality', 'Discount', 'Payment_m
ode', 'Sale_data']

```
In [23]: print(df.columns.tolist())
```

```
['Transaction_id', 'Customer_id', 'Product_id', 'Quality', 'Discount', 'Payment_m  
ode', 'Sale_data']
```

```
In [24]: # Program 8.2 – Create X and y
```

```
# Target variable
y = df["Quality"]

# Input features
X = df.drop(columns=["Quality"])

# Remove date column
if "Sale_data" in X.columns:
    X = X.drop(columns=["Sale_data"])

print("X shape:", X.shape)
print("y shape:", y.shape)

print("\nX columns:")
print(X.columns.tolist())
```

X shape: (25, 5)
y shape: (25,)

X columns:
['Transaction_id', 'Customer_id', 'Product_id', 'Discount', 'Payment_mode']

```
In [25]: import os

for root, dirs, files in os.walk(
    r"C:\Users\yaswa\Desktop\Sales_MLOps\data"
):
    for file in files:
        print(os.path.join(root, file))
```

```
C:\Users\yaswa\Desktop\Sales_MLOps\data\raw\.gitignore
C:\Users\yaswa\Desktop\Sales_MLOps\data\raw\sales_data.csv
C:\Users\yaswa\Desktop\Sales_MLOps\data\raw\sales_data.csv.dvc
```

```
In [27]: #Program 8.3 – Train/Test Split
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)

print("Training data:", X_train.shape)
print("Testing data :", X_test.shape)

print("Training target:", y_train.shape)
print("Testing target :", y_test.shape)
```

```
Training data: (20, 5)
Testing data : (5, 5)
Training target: (20,)
Testing target : (5,)
```

```
In [31]: import pandas as pd

df = pd.read_csv(
    r"C:\Users\yaswa\Desktop\Sales_MLOps\data\raw\sales_data.csv"
)

print("Columns in dataset:")
print(df.columns.tolist())
```

```
Columns in dataset:
['Transaction_id', 'Customer_id', 'Product_id', 'Quality', 'Discount', 'Payment_m
ode', 'Sale_data']
```

```
In [32]: print(df.head(10))
print("\nData types:")
print(df.dtypes)
print("\nUnique values:")
for col in df.columns:
    print("\n", col)
    print(df[col].unique()[:10])
```

	Transaction_id	Customer_id	Product_id	Quality	Discount	Payment_mode	\
0	T001	C001	P001	2	5	UPI	
1	T002	C002	P002	3	10	Cash	
2	T003	C003	P003	1	0	Card	
3	T004	C004	P004	2	15	UPI	
4	T005	C005	P005	1	5	Cash	
5	T006	C006	P006	5	10	Card	
6	T007	C007	P007	2	0	UPI	
7	T008	C008	P008	3	5	Cash	
8	T009	C009	P009	1	10	Card	
9	T010	C010	P010	2	15	UPI	

Sale_data

0	01-07-2026
1	02-07-2026
2	03-07-2026
3	04-07-2026
4	05-07-2026
5	06-07-2026
6	07-07-2026
7	08-07-2026
8	09-07-2026
9	10-07-2026

Data types:

Transaction_id	object
Customer_id	object
Product_id	object
Quality	int64
Discount	int64
Payment_mode	object
Sale_data	object

dtype: object

Unique values:

Transaction_id

['T001' 'T002' 'T003' 'T004' 'T005' 'T006' 'T007' 'T008' 'T009' 'T010']

Customer_id

['C001' 'C002' 'C003' 'C004' 'C005' 'C006' 'C007' 'C008' 'C009' 'C010']

Product_id

['P001' 'P002' 'P003' 'P004' 'P005' 'P006' 'P007' 'P008' 'P009' 'P010']

Quality

[2 3 1 5 4]

Discount

[5 10 0 15]

Payment_mode

['UPI' 'Cash' 'Card']

Sale_data

['01-07-2026' '02-07-2026' '03-07-2026' '04-07-2026' '05-07-2026'
'06-07-2026' '07-07-2026' '08-07-2026' '09-07-2026' '10-07-2026']

In [33]: `print(df["Sale_data"].head(10))`

```
0    01-07-2026
1    02-07-2026
2    03-07-2026
3    04-07-2026
4    05-07-2026
5    06-07-2026
6    07-07-2026
7    08-07-2026
8    09-07-2026
9    10-07-2026
Name: Sale_data, dtype: object
```

```
In [37]: import pandas as pd

df = pd.read_csv(
    r"C:\Users\yaswa\Desktop\Sales_MLOps\data\raw\sales_data.csv"
)

print("=====")
print("DATASET INFORMATION")
print("=====")

print("\nShape:")
print(df.shape)

print("\nColumns:")
print(df.columns.tolist())

print("\nFirst 10 rows:")
print(df.head(10))

print("\nData Types:")
print(df.dtypes)

print("\nUnique values:")
for col in df.columns:
    print("\n" + col)
    print(df[col].unique()[:10])
```



```
=====
DATASET INFORMATION
=====
```

Shape:
(25, 7)

Columns:
['Transaction_id', 'Customer_id', 'Product_id', 'Quality', 'Discount', 'Payment_m
ode', 'Sale_data']

First 10 rows:

	Transaction_id	Customer_id	Product_id	Quality	Discount	Payment_mode \
0	T001	C001	P001	2	5	UPI
1	T002	C002	P002	3	10	Cash
2	T003	C003	P003	1	0	Card
3	T004	C004	P004	2	15	UPI
4	T005	C005	P005	1	5	Cash
5	T006	C006	P006	5	10	Card
6	T007	C007	P007	2	0	UPI
7	T008	C008	P008	3	5	Cash
8	T009	C009	P009	1	10	Card
9	T010	C010	P010	2	15	UPI

	Sale_data
0	01-07-2026
1	02-07-2026
2	03-07-2026
3	04-07-2026
4	05-07-2026
5	06-07-2026
6	07-07-2026
7	08-07-2026
8	09-07-2026
9	10-07-2026

Data Types:

Transaction_id	object
Customer_id	object
Product_id	object
Quality	int64
Discount	int64
Payment_mode	object
Sale_data	object

dtype: object

Unique values:

Transaction_id
['T001' 'T002' 'T003' 'T004' 'T005' 'T006' 'T007' 'T008' 'T009' 'T010']

Customer_id
['C001' 'C002' 'C003' 'C004' 'C005' 'C006' 'C007' 'C008' 'C009' 'C010']

Product_id
['P001' 'P002' 'P003' 'P004' 'P005' 'P006' 'P007' 'P008' 'P009' 'P010']

Quality
[2 3 1 5 4]

Discount

[5 10 0 15]

Payment_mode

['UPI' 'Cash' 'Card']

Sale_data

['01-07-2026' '02-07-2026' '03-07-2026' '04-07-2026' '05-07-2026'
'06-07-2026' '07-07-2026' '08-07-2026' '09-07-2026' '10-07-2026']

```
In [38]: #Stage 9 - Preprocessing Pipeline
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer

from sklearn.impute import SimpleImputer
from sklearn.preprocessing import (
    StandardScaler,
    OneHotEncoder
)

# =====
# STAGE 9 - PREPROCESSING
# =====

# Numerical columns
numerical_features = X.select_dtypes(
    include=["int64", "float64"]
).columns.tolist()

# Categorical columns
categorical_features = X.select_dtypes(
    include=["object", "category", "bool"]
).columns.tolist()

print("Numerical features:")
print(numerical_features)

print("\nCategorical features:")
print(categorical_features)

# Numerical pipeline
numerical_pipeline = Pipeline(
    steps=[
        (
            "imputer",
            SimpleImputer(
                strategy="median"
            )
        ),
        (
            "scaler",
            StandardScaler()
        )
    ]
)

# Categorical pipeline
categorical_pipeline = Pipeline(
```

```

steps=[
    (
        "imputer",
        SimpleImputer(
            strategy="most_frequent"
        )
    ),
    (
        "encoder",
        OneHotEncoder(
            handle_unknown="ignore"
        )
    )
]

# ColumnTransformer
preprocessor = ColumnTransformer(
    transformers=[
        (
            "numerical",
            numerical_pipeline,
            numerical_features
        ),
        (
            "categorical",
            categorical_pipeline,
            categorical_features
        )
    ]
)

print("\n=====")
print("STAGE 9 COMPLETED")
print("=====")

print("Preprocessing pipeline created successfully.")

```

Numerical features:

['Discount']

Categorical features:

['Transaction_id', 'Customer_id', 'Product_id', 'Payment_mode']

=====

STAGE 9 COMPLETED

=====

Preprocessing pipeline created successfully.

```

In [39]: #Stage 10 – Random Forest + Evaluation
import numpy as np

from sklearn.ensemble import RandomForestRegressor

from sklearn.pipeline import Pipeline

from sklearn.metrics import (
    mean_absolute_error,
    mean_squared_error,

```

```
    r2_score
)

# =====
# STAGE 10 - MODEL TRAINING
# =====

# Create model
model = RandomForestRegressor(
    n_estimators=100,
    random_state=42
)

# Complete pipeline
model_pipeline = Pipeline(
    steps=[
        (
            "preprocessor",
            preprocessor
        ),
        (
            "model",
            model
        )
    ]
)

# Train
model_pipeline.fit(
    X_train,
    y_train
)

print("Model trained successfully.")

# Predict
y_pred = model_pipeline.predict(
    X_test
)

# MAE
mae = mean_absolute_error(
    y_test,
    y_pred
)

# RMSE
rmse = np.sqrt(
    mean_squared_error(
        y_test,
        y_pred
    )
)

# R2
r2 = r2_score(
```

```
    y_test,\n    y_pred\n)\n\n# Results\nprint("\\n=====")\nprint("STAGE 10 - MODEL EVALUATION")\nprint("=====")\n\nprint("MAE :", mae)\nprint("RMSE:", rmse)\nprint("R2  :", r2)
```

Model trained successfully.

```
=====\nSTAGE 10 - MODEL EVALUATION\n=====\nMAE : 0.8060000000000003\nRMSE: 0.9328987083279728\nR2  : -1.1757500000000003
```

```
In [43]: #Stage 11 - MLflow\n!pip install mlflow
```

Requirement already satisfied: mlflow in c:\users\yaswa\anaconda3\lib\site-packages (3.15.1)

Requirement already satisfied: mlflow-skinny==3.15.1 in c:\users\yaswa\anaconda3\lib\site-packages (from mlflow) (3.15.1)

Requirement already satisfied: mlflow-tracing==3.15.1 in c:\users\yaswa\anaconda3\lib\site-packages (from mlflow) (3.15.1)

Requirement already satisfied: Flask-CORS<7 in c:\users\yaswa\anaconda3\lib\site-packages (from mlflow) (6.0.5)

Requirement already satisfied: Flask<4 in c:\users\yaswa\anaconda3\lib\site-packages (from mlflow) (3.0.3)

Requirement already satisfied: aiohttp<4,>=3.7.0 in c:\users\yaswa\anaconda3\lib\site-packages (from mlflow) (3.10.5)

Requirement already satisfied: alembic!=1.10.0,<2 in c:\users\yaswa\anaconda3\lib\site-packages (from mlflow) (1.19.1)

Requirement already satisfied: cryptography<50,>=43.0.0 in c:\users\yaswa\anaconda3\lib\site-packages (from mlflow) (43.0.0)

Requirement already satisfied: docker<8,>=4.0.0 in c:\users\yaswa\anaconda3\lib\site-packages (from mlflow) (7.2.0)

Requirement already satisfied: graphene<4 in c:\users\yaswa\anaconda3\lib\site-packages (from mlflow) (3.4.3)

Requirement already satisfied: huey<4,>=2.5.4 in c:\users\yaswa\anaconda3\lib\site-packages (from mlflow) (3.3.4)

Requirement already satisfied: matplotlib<4 in c:\users\yaswa\anaconda3\lib\site-packages (from mlflow) (3.9.2)

Requirement already satisfied: numpy<3 in c:\users\yaswa\anaconda3\lib\site-packages (from mlflow) (1.26.4)

Requirement already satisfied: pandas<3 in c:\users\yaswa\anaconda3\lib\site-packages (from mlflow) (2.2.2)

Requirement already satisfied: pyarrow<26,>=4.0.0 in c:\users\yaswa\anaconda3\lib\site-packages (from mlflow) (16.1.0)

Requirement already satisfied: scikit-learn<2 in c:\users\yaswa\anaconda3\lib\site-packages (from mlflow) (1.5.1)

Requirement already satisfied: scipy<2 in c:\users\yaswa\anaconda3\lib\site-packages (from mlflow) (1.13.1)

Requirement already satisfied: skops<1 in c:\users\yaswa\anaconda3\lib\site-packages (from mlflow) (0.14.0)

Requirement already satisfied: sqlalchemy<3,>=1.4.0 in c:\users\yaswa\anaconda3\lib\site-packages (from mlflow) (2.0.34)

Requirement already satisfied: waitress<4 in c:\users\yaswa\anaconda3\lib\site-packages (from mlflow) (3.0.2)

Requirement already satisfied: cachetools<8,>=5.0.0 in c:\users\yaswa\anaconda3\lib\site-packages (from mlflow-skinny==3.15.1->mlflow) (5.3.3)

Requirement already satisfied: click<9,>=7.0 in c:\users\yaswa\anaconda3\lib\site-packages (from mlflow-skinny==3.15.1->mlflow) (8.1.7)

Requirement already satisfied: cloudpickle<4 in c:\users\yaswa\anaconda3\lib\site-packages (from mlflow-skinny==3.15.1->mlflow) (3.0.0)

Requirement already satisfied: databricks-sdk<1,>=0.20.0 in c:\users\yaswa\anaconda3\lib\site-packages (from mlflow-skinny==3.15.1->mlflow) (0.133.0)

Requirement already satisfied: fastapi<1 in c:\users\yaswa\anaconda3\lib\site-packages (from mlflow-skinny==3.15.1->mlflow) (0.141.1)

Requirement already satisfied: gitpython<4,>=3.1.9 in c:\users\yaswa\anaconda3\lib\site-packages (from mlflow-skinny==3.15.1->mlflow) (3.1.43)

Requirement already satisfied: importlib_metadata!=4.7.0,<10,>=3.7.0 in c:\users\yaswa\anaconda3\lib\site-packages (from mlflow-skinny==3.15.1->mlflow) (7.0.1)

Requirement already satisfied: opentelemetry-api<3,>=1.9.0 in c:\users\yaswa\anaconda3\lib\site-packages (from mlflow-skinny==3.15.1->mlflow) (1.44.0)

Requirement already satisfied: opentelemetry-proto<3,>=1.9.0 in c:\users\yaswa\anaconda3\lib\site-packages (from mlflow-skinny==3.15.1->mlflow) (1.44.0)

Requirement already satisfied: opentelemetry-sdk<3,>=1.9.0 in c:\users\yaswa\anaconda3\lib\site-packages (from mlflow-skinny==3.15.1->mlflow) (1.44.0)

Requirement already satisfied: packaging<27 in c:\users\yaswa\anaconda3\lib\site-packages (from mlflow-skinny==3.15.1->mlflow) (24.1)

Requirement already satisfied: protobuf<8,>=3.12.0 in c:\users\yaswa\anaconda3\lib\site-packages (from mlflow-skinny==3.15.1->mlflow) (6.33.6)

Requirement already satisfied: pydantic<3,>=2.0.0 in c:\users\yaswa\anaconda3\lib\site-packages (from mlflow-skinny==3.15.1->mlflow) (2.13.4)

Requirement already satisfied: python-dotenv<2,>=0.19.0 in c:\users\yaswa\anaconda3\lib\site-packages (from mlflow-skinny==3.15.1->mlflow) (0.21.0)

Requirement already satisfied: pyyaml<7,>=5.1 in c:\users\yaswa\anaconda3\lib\site-packages (from mlflow-skinny==3.15.1->mlflow) (6.0.3)

Requirement already satisfied: requests<3,>=2.17.3 in c:\users\yaswa\anaconda3\lib\site-packages (from mlflow-skinny==3.15.1->mlflow) (2.32.3)

Requirement already satisfied: sqlparse<1,>=0.4.0 in c:\users\yaswa\anaconda3\lib\site-packages (from mlflow-skinny==3.15.1->mlflow) (0.6.0)

Requirement already satisfied: starlette<2 in c:\users\yaswa\anaconda3\lib\site-packages (from mlflow-skinny==3.15.1->mlflow) (1.6.0)

Requirement already satisfied: typing-extensions<5,>=4.0.0 in c:\users\yaswa\anaconda3\lib\site-packages (from mlflow-skinny==3.15.1->mlflow) (4.15.0)

Requirement already satisfied: uvicorn<1 in c:\users\yaswa\anaconda3\lib\site-packages (from mlflow-skinny==3.15.1->mlflow) (0.52.4)

Requirement already satisfied: aiohappyeyeballs>=2.3.0 in c:\users\yaswa\anaconda3\lib\site-packages (from aiohttp<4,>=3.7.0->mlflow) (2.4.0)

Requirement already satisfied: aiosignal>=1.1.2 in c:\users\yaswa\anaconda3\lib\site-packages (from aiohttp<4,>=3.7.0->mlflow) (1.2.0)

Requirement already satisfied: attrs>=17.3.0 in c:\users\yaswa\anaconda3\lib\site-packages (from aiohttp<4,>=3.7.0->mlflow) (23.1.0)

Requirement already satisfied: frozenlist>=1.1.1 in c:\users\yaswa\anaconda3\lib\site-packages (from aiohttp<4,>=3.7.0->mlflow) (1.4.0)

Requirement already satisfied: multidict<7.0,>=4.5 in c:\users\yaswa\anaconda3\lib\site-packages (from aiohttp<4,>=3.7.0->mlflow) (6.0.4)

Requirement already satisfied: yarl<2.0,>=1.0 in c:\users\yaswa\anaconda3\lib\site-packages (from aiohttp<4,>=3.7.0->mlflow) (1.11.0)

Requirement already satisfied: Mako in c:\users\yaswa\anaconda3\lib\site-packages (from alembic!=1.10.0,<2->mlflow) (1.4.1)

Requirement already satisfied: cffi>=1.12 in c:\users\yaswa\anaconda3\lib\site-packages (from cryptography<50,>=43.0.0->mlflow) (1.17.1)

Requirement already satisfied: pywin32>=304 in c:\users\yaswa\anaconda3\lib\site-packages (from docker<8,>=4.0.0->mlflow) (305.1)

Requirement already satisfied: urllib3>=1.26.0 in c:\users\yaswa\anaconda3\lib\site-packages (from docker<8,>=4.0.0->mlflow) (2.2.3)

Requirement already satisfied: Werkzeug>=3.0.0 in c:\users\yaswa\anaconda3\lib\site-packages (from Flask<4->mlflow) (3.0.3)

Requirement already satisfied: Jinja2>=3.1.2 in c:\users\yaswa\anaconda3\lib\site-packages (from Flask<4->mlflow) (3.1.6)

Requirement already satisfied: itsdangerous>=2.1.2 in c:\users\yaswa\anaconda3\lib\site-packages (from Flask<4->mlflow) (2.2.0)

Requirement already satisfied: blinker>=1.6.2 in c:\users\yaswa\anaconda3\lib\site-packages (from Flask<4->mlflow) (1.6.2)

Requirement already satisfied: graphql-core<3.3,>=3.1 in c:\users\yaswa\anaconda3\lib\site-packages (from graphene<4->mlflow) (3.2.11)

Requirement already satisfied: graphql-relay<3.3,>=3.1 in c:\users\yaswa\anaconda3\lib\site-packages (from graphene<4->mlflow) (3.2.0)

Requirement already satisfied: python-dateutil<3,>=2.7.0 in c:\users\yaswa\anaconda3\lib\site-packages (from graphene<4->mlflow) (2.9.0.post0)

Requirement already satisfied: contourpy>=1.0.1 in c:\users\yaswa\anaconda3\lib\site-packages (from matplotlib<4->mlflow) (1.2.0)

Requirement already satisfied: cycler>=0.10 in c:\users\yaswa\anaconda3\lib\site-packages (from matplotlib<4->mlflow) (0.11.0)

Requirement already satisfied: fonttools>=4.22.0 in c:\users\yaswa\anaconda3\lib\site-packages (from matplotlib<4->mlflow) (4.51.0)

Requirement already satisfied: kiwisolver>=1.3.1 in c:\users\yaswa\anaconda3\lib\site-packages (from matplotlib<4->mlflow) (1.4.4)

Requirement already satisfied: pillow>=8 in c:\users\yaswa\anaconda3\lib\site-packages (from matplotlib<4->mlflow) (10.4.0)

Requirement already satisfied: pyparsing>=2.3.1 in c:\users\yaswa\anaconda3\lib\site-packages (from matplotlib<4->mlflow) (3.1.2)

Requirement already satisfied: pytz>=2020.1 in c:\users\yaswa\anaconda3\lib\site-packages (from pandas<3->mlflow) (2024.1)

Requirement already satisfied: tzdata>=2022.7 in c:\users\yaswa\anaconda3\lib\site-packages (from pandas<3->mlflow) (2023.3)

Requirement already satisfied: joblib>=1.2.0 in c:\users\yaswa\anaconda3\lib\site-packages (from scikit-learn<2->mlflow) (1.4.2)

Requirement already satisfied: threadpoolctl>=3.1.0 in c:\users\yaswa\anaconda3\lib\site-packages (from scikit-learn<2->mlflow) (3.5.0)

Requirement already satisfied: prettytable>=3.9 in c:\users\yaswa\anaconda3\lib\site-packages (from skops<1->mlflow) (3.18.0)

Requirement already satisfied: greenlet!=0.4.17 in c:\users\yaswa\anaconda3\lib\site-packages (from sqlalchemy<3,>=1.4.0->mlflow) (3.0.1)

Requirement already satisfied: pycparser in c:\users\yaswa\anaconda3\lib\site-packages (from cffi>=1.12->cryptography<50,>=43.0.0->mlflow) (2.21)

Requirement already satisfied: colorama in c:\users\yaswa\anaconda3\lib\site-packages (from click<9,>=7.0->mlflow-skinny==3.15.1->mlflow) (0.4.6)

Requirement already satisfied: google-auth~=2.0 in c:\users\yaswa\anaconda3\lib\site-packages (from databricks-sdk<1,>=0.20.0->mlflow-skinny==3.15.1->mlflow) (2.42.1)

Requirement already satisfied: typing-inspection>=0.4.2 in c:\users\yaswa\anaconda3\lib\site-packages (from fastapi<1->mlflow-skinny==3.15.1->mlflow) (0.4.4)

Requirement already satisfied: annotated-doc>=0.0.2 in c:\users\yaswa\anaconda3\lib\site-packages (from fastapi<1->mlflow-skinny==3.15.1->mlflow) (0.0.5)

Requirement already satisfied: gitdb<5,>=4.0.1 in c:\users\yaswa\anaconda3\lib\site-packages (from gitpython<4,>=3.1.9->mlflow-skinny==3.15.1->mlflow) (4.0.7)

Requirement already satisfied: zipp>=0.5 in c:\users\yaswa\anaconda3\lib\site-packages (from importlib_metadata!=4.7.0,<10,>=3.7.0->mlflow-skinny==3.15.1->mlflow) (3.17.0)

Requirement already satisfied: MarkupSafe>=2.0 in c:\users\yaswa\anaconda3\lib\site-packages (from Jinja2>=3.1.2->Flask<4->mlflow) (2.1.3)

Requirement already satisfied: opentelemetry-semantic-conventions==0.65b0 in c:\users\yaswa\anaconda3\lib\site-packages (from opentelemetry-sdk<3,>=1.9.0->mlflow-skinny==3.15.1->mlflow) (0.65b0)

Requirement already satisfied: wcwidth>=0.3.5 in c:\users\yaswa\anaconda3\lib\site-packages (from prettytable>=3.9->skops<1->mlflow) (0.8.2)

Requirement already satisfied: annotated-types>=0.6.0 in c:\users\yaswa\anaconda3\lib\site-packages (from pydantic<3,>=2.0.0->mlflow-skinny==3.15.1->mlflow) (0.6.0)

Requirement already satisfied: pydantic-core==2.46.4 in c:\users\yaswa\anaconda3\lib\site-packages (from pydantic<3,>=2.0.0->mlflow-skinny==3.15.1->mlflow) (2.46.4)

Requirement already satisfied: six>=1.5 in c:\users\yaswa\anaconda3\lib\site-packages (from python-dateutil<3,>=2.7.0->graphene<4->mlflow) (1.16.0)

Requirement already satisfied: charset-normalizer<4,>=2 in c:\users\yaswa\anaconda3\lib\site-packages (from requests<3,>=2.17.3->mlflow-skinny==3.15.1->mlflow) (3.3.2)

Requirement already satisfied: idna<4,>=2.5 in c:\users\yaswa\anaconda3\lib\site-packages (from requests<3,>=2.17.3->mlflow-skinny==3.15.1->mlflow) (3.7)

Requirement already satisfied: certifi>=2017.4.17 in c:\users\yaswa\anaconda3\lib\site-packages (from requests<3,>=2.17.3->mlflow-skinny==3.15.1->mlflow) (2024.8.30)

Requirement already satisfied: anyio<5,>=3.6.2 in c:\users\yaswa\anaconda3\lib\site-packages (from starlette<2->mlflow-skinny==3.15.1->mlflow) (4.2.0)

Requirement already satisfied: h11>=0.8 in c:\users\yaswa\anaconda3\lib\site-pack

ages (from uvicorn<1->mlflow-skinny==3.15.1->mlflow) (0.14.0)
 Requirement already satisfied: sniffio>=1.1 in c:\users\yaswa\anaconda3\lib\site-packages (from anyio<5,>=3.6.2->starlette<2->mlflow-skinny==3.15.1->mlflow) (1.3.0)
 Requirement already satisfied: smmap<5,>=3.0.1 in c:\users\yaswa\anaconda3\lib\site-packages (from gitdb<5,>=4.0.1->gitpython<4,>=3.1.9->mlflow-skinny==3.15.1->mlflow) (4.0.0)
 Requirement already satisfied: pyasn1-modules>=0.2.1 in c:\users\yaswa\anaconda3\lib\site-packages (from google-auth~=2.0->databricks-sdk<1,>=0.20.0->mlflow-skinny==3.15.1->mlflow) (0.2.8)
 Requirement already satisfied: rsa<5,>=3.1.4 in c:\users\yaswa\anaconda3\lib\site-packages (from google-auth~=2.0->databricks-sdk<1,>=0.20.0->mlflow-skinny==3.15.1->mlflow) (4.9.1)
 Requirement already satisfied: pyasn1<0.5.0,>=0.4.6 in c:\users\yaswa\anaconda3\lib\site-packages (from pyasn1-modules>=0.2.1->google-auth~=2.0->databricks-sdk<1,>=0.20.0->mlflow-skinny==3.15.1->mlflow) (0.4.8)

```
In [45]: import mlflow
import mlflow.sklearn
import numpy as np

from sklearn.metrics import (
    mean_absolute_error,
    mean_squared_error,
    r2_score
)

# =====
# STAGE 11 - MLFLOW
# =====

mlflow.set_experiment(
    "Sales_Amount_Prediction"
)

with mlflow.start_run():

    # -----
    # Train model
    # -----
    model_pipeline.fit(
        X_train,
        y_train
    )

    # -----
    # Prediction
    # -----
    y_pred = model_pipeline.predict(
        X_test
    )

    # -----
    # Metrics
    # -----
    mae = mean_absolute_error(
        y_test,
        y_pred
    )
```

```
rmse = np.sqrt(
    mean_squared_error(
        y_test,
        y_pred
    )
)

r2 = r2_score(
    y_test,
    y_pred
)

# -----
# Parameters
# -----
mlflow.log_param(
    "model",
    "RandomForestRegressor"
)

mlflow.log_param(
    "n_estimators",
    100
)

mlflow.log_param(
    "random_state",
    42
)

# -----
# Metrics
# -----
mlflow.log_metric(
    "MAE",
    mae
)

mlflow.log_metric(
    "RMSE",
    rmse
)

mlflow.log_metric(
    "R2",
    r2
)

# -----
# Save model to MLflow
# -----
mlflow.sklearn.log_model(
    sk_model=model_pipeline,
    name="sales_prediction_model",
    skops_trusted_types=[
        "numpy.dtype",
        "sklearn.compose._column_transformer._RemainderColsList"
    ]
)
```

```
# -----  
# Result  
# -----  
print("\n=====")  
print("STAGE 11 COMPLETED")  
print("=====")  
  
print("MAE :", mae)  
print("RMSE:", rmse)  
print("R2  :", r2)
```

```
=====  
STAGE 11 COMPLETED  
=====  
MAE : 0.8060000000000003  
RMSE: 0.9328987083279728  
R2  : -1.1757500000000003
```

In []: